Graphs

Data Structures and Algorithms

Housekeeping:

- Midterms **next** week. Thursday.
  - Check your seat and venue!
  - Midterm announcement on Coursemology has details.
  - Past years released.

Plan for Today:

- Introduction to graphs and basic graph algorithms
- Basic graph data structures

Recall from CS1231S:

Graphs: Definitions and Basic Properties

Definition: Undirected Graph

An undirected **graph** G consists of 2 finite sets: a nonempty set V of **vertices** and a set E of **edges**, where each (undirected) edge is associated with a set consisting of either one or two vertices called its **endpoints**.

An edge is said to **connect** its endpoints; two vertices that are connected by an edge are called **adjacent vertices**; and a vertex that is an endpoint of a loop is said to be **adjacent to itself**.

An edge is said to be **incident on** each of its endpoints, and two edges incident on the same endpoint are called **adjacent edges**.

We write $e = \{v, w\}$ for an undirected edge e incident on vertices v and w.

Core simplifying assumption 1:

For our graphs, we're going to assume that the n nodes/vertices come in running IDs starting from **0 through n - 1**.

This makes using our data structures a lot easier and efficient.

(... and so we don't need to use a hashtable that complicates the whole analysis)

Core simplifying assumption 2:

Unless otherwise specified, our graphs may be directed/undirected, and weighted/unweighted.

But by default they are also **simple graphs**. Unless otherwise specified.

But how do we store vertices/edges?

Graph Data Structures

How to store graphs in computers

After all this you might realise that the only good one is adjacency lists… but anyway…

But how do we store vertices/edges?

There are 3 main ways:

- Adjacency matrices
- Adjacency lists
- Edge lists

Typical operations to support:

1. `is_adjacent(i, j)`
   - Returns whether there is an edge **from** node i **to** node j and the weight (if any). (This is one-directional)

2. `get_neighbours(i)`
   - Returns an array of all the neighbouring nodes (and weights to those neighbours (if any).

3. `in_degree_of(i)`
   - Returns the in-degree of a node i.

4. `out_degree_of(i)`
   - Returns the out-degree of a node i

Adjacency Matrix:

Given **n** nodes and **m** edges, we store it as an array `arr` of **n** arrays.

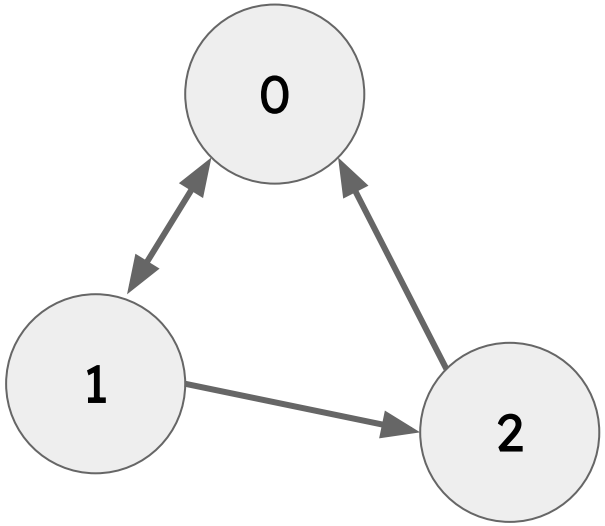
Each array is **n** elements long.

- If the graph stores unweighted edges, then the elements are 0/1.
- Otherwise the graph is weighted, then the elements store the weights.

- `arr[i][j] = 0` if there is no edge from node i to node j.
- Otherwise, `arr[i][j]` is equal to 1 for unweighted graphs, or equal to **w** for the weight.

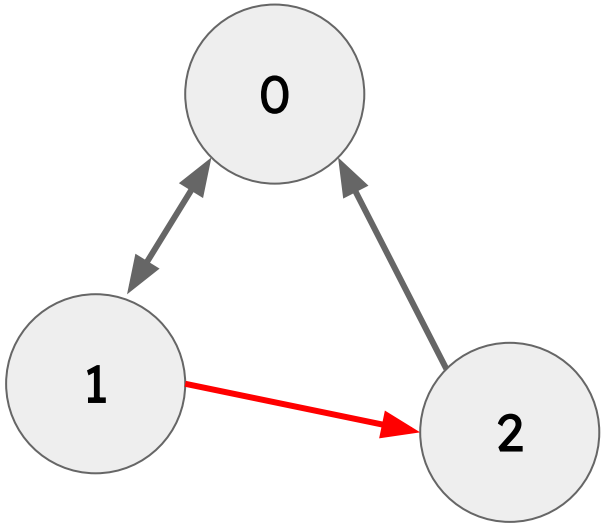
Adjacency Matrix:

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$



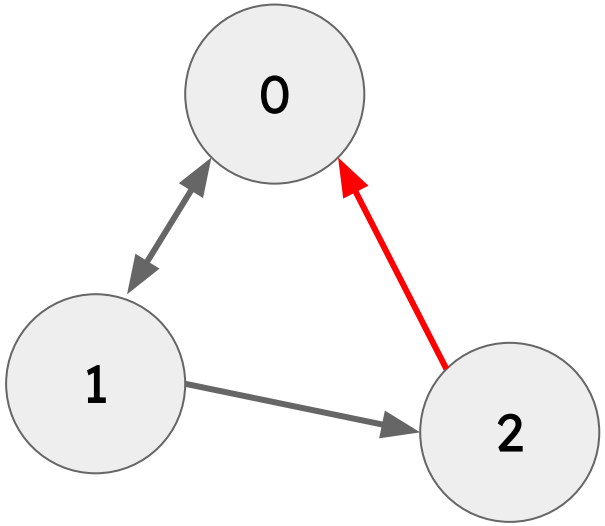
Adjacency Matrix:

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$



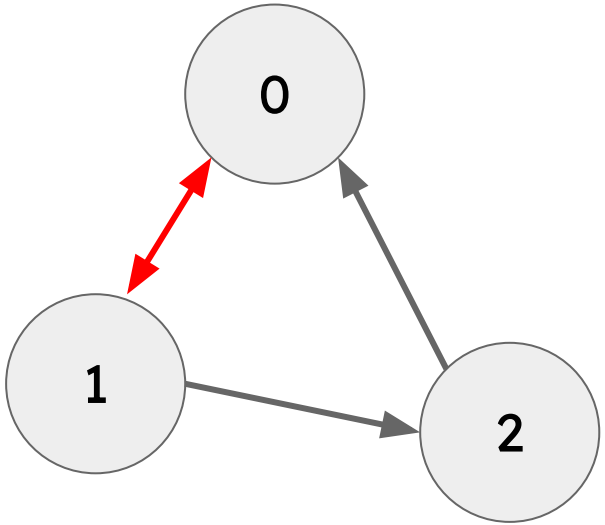
Adjacency Matrix:

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$



Adjacency Matrix:

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$



Adjacency Matrix:

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & \boxed{1} \\ 1 & 0 & 0 \end{pmatrix}$$

is_adjacent(1, 2)

Just lookup mat[i][j].

Adjacency Matrix:

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

$$\begin{pmatrix} 0 & 1 & 0 \\ \boxed{1 \quad 0 \quad 1} \\ 1 & 0 & 0 \end{pmatrix}$$

get_neighbours(1)

Iterate on mat[i] and filter out indices j for which mat[i][j] != 0

Adjacency Matrix:

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

in_degree_of(2)

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Count the number of indices j such that mat[j][i] != 0.

Adjacency Matrix:

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

out_degree_of(0)

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Count the number of indices j such that mat[i][j] != 0.

Adjacency Matrix:

	Adjacency Matrix		
is_adjacent	O(1)		
get_neighbours	O(n)		
in_degree_of	O(n)		
out_degree_of	O(n)		

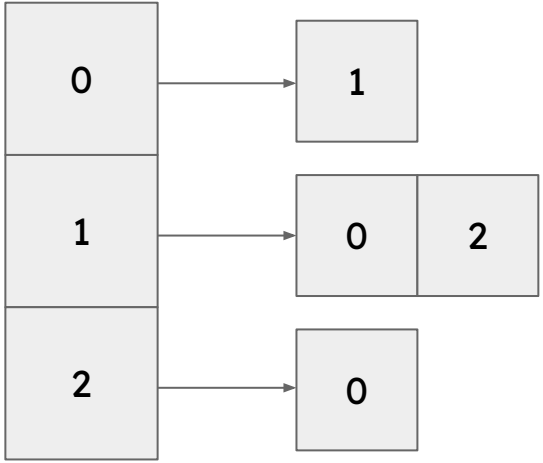
Adjacency List:

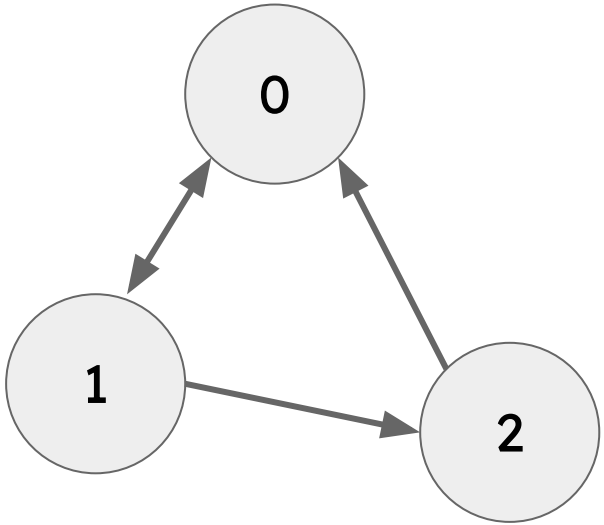
Given **n** nodes and **m** edges, we store it as an array `arr` of **n** arrays.

The ith array `arr[i]` is deg(i) long and only stores edges (and weights, if need be) to nodes that it is adjacent to.

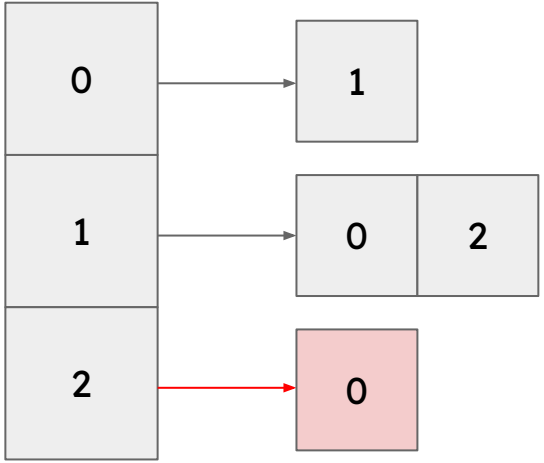
If there is an edge from node i to node j, then j is stored in the array `arr[i]`.

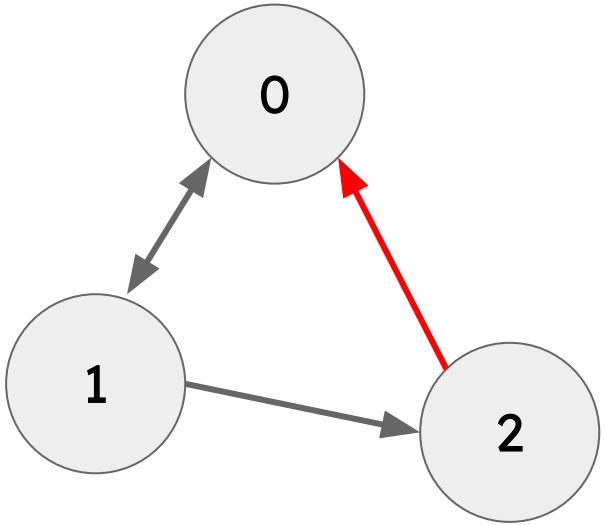
Adjacency List:



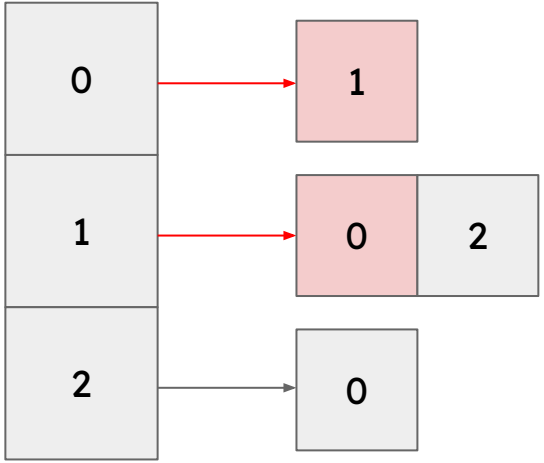


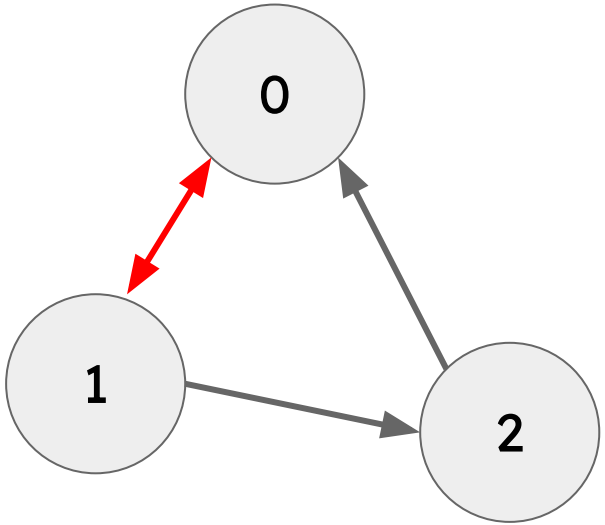
Adjacency List:



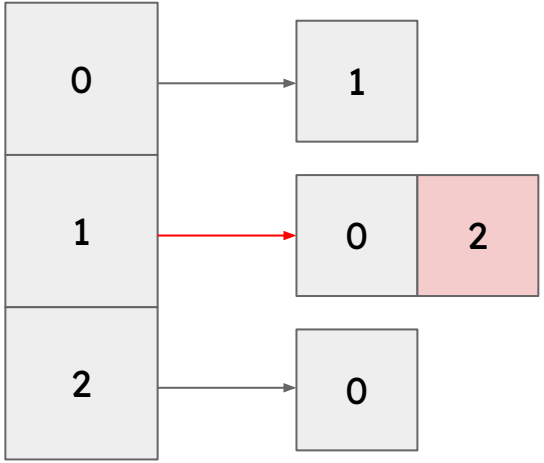


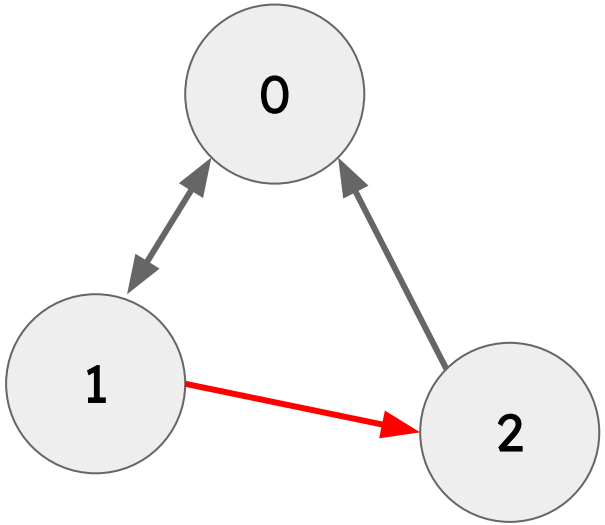
Adjacency List:





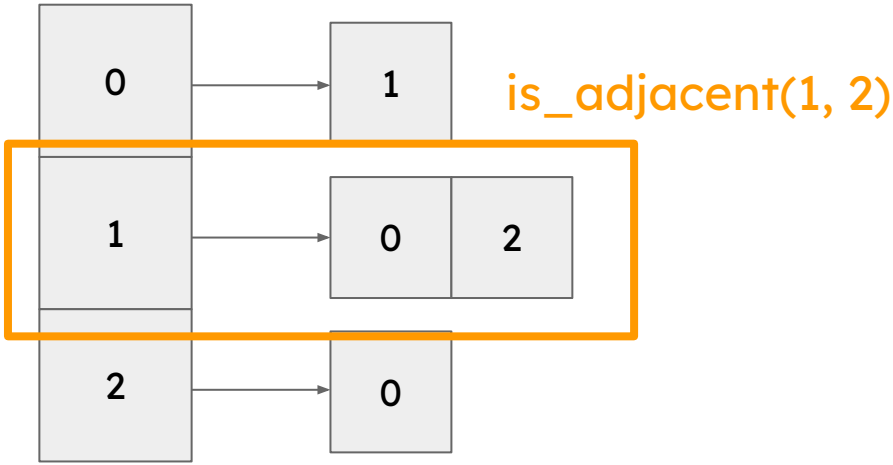
Adjacency List:





Adjacency List:

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)



Need to look for j in arr[i].

If arr[i] is sorted, then

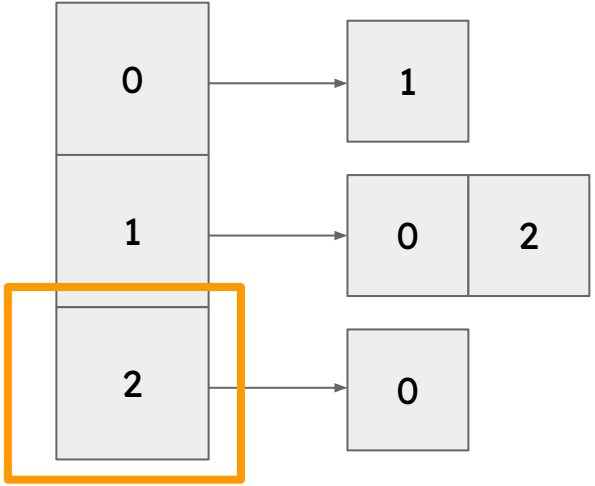
O(log deg(i))

Otherwise,

O(deg(i))

Adjacency List:

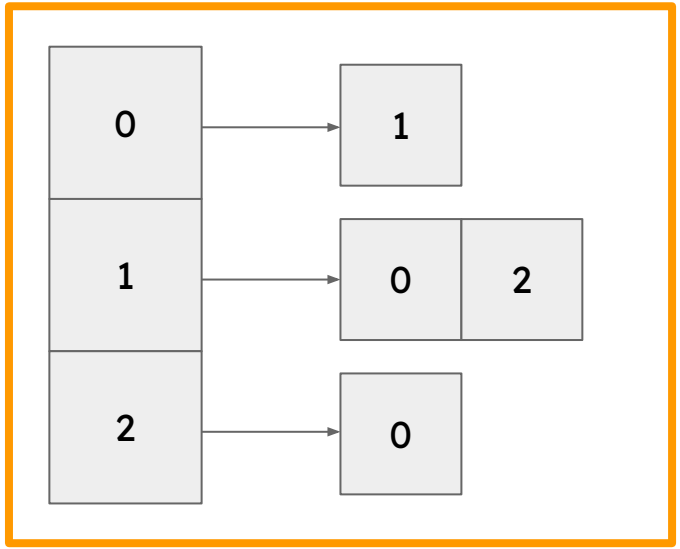
1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)



get_neighbours(2)

Just return arr[i]

Adjacency List:



in_degree_of(0)

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

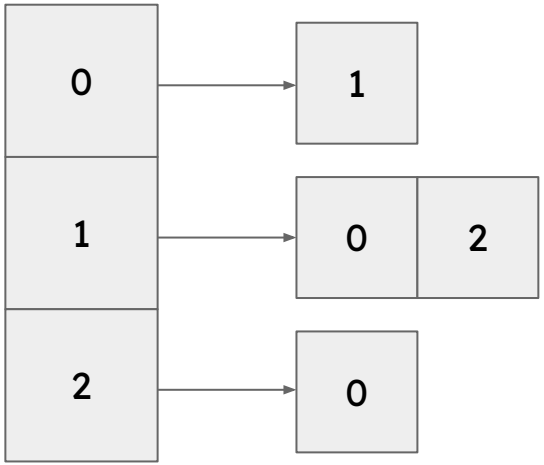
Need to go through all the nodes and see which point to node i.

If lists are not sorted, O(n + m)

- O(n) to go through each node
- O(m) to go through all the edges.

Adjacency List:

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)



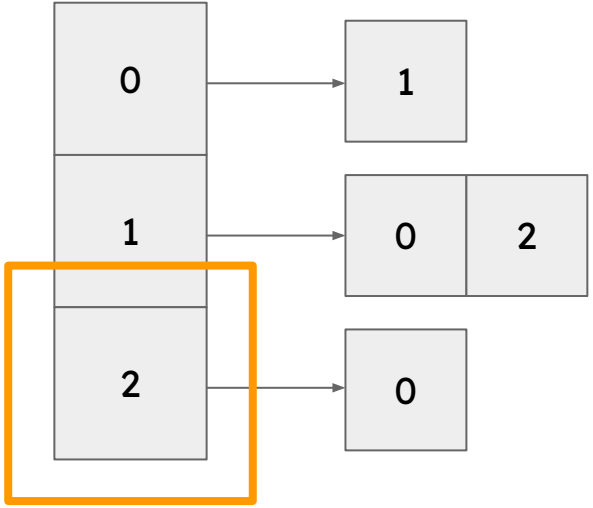
Need to go through all the nodes and see which point to node i.

If lists are sorted, O(n log(m/n))

Challenge:
Prove if the lists are sorted,
the runtime is O(n log (m/n))

Something to Try

Adjacency List:



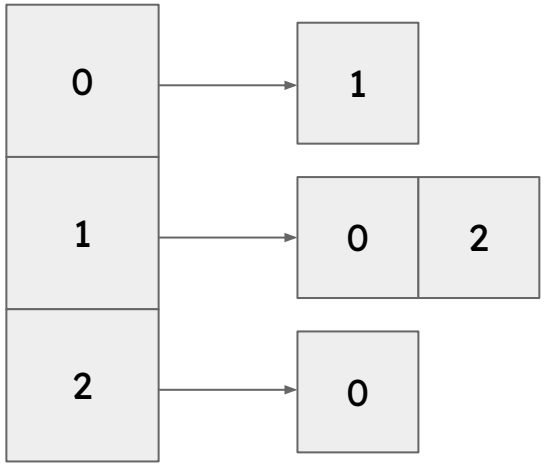
out_degree_of(2)

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

Just return the size of arr[i]

Adjacency List:

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)



By default we don't assume the neighbour lists are sorted.

(There's typically little reason to assume so, and it requires the nodes to come with comparable IDs)

Complexities:

	Adjacency Matrix	Adjacency List	
is_adjacent	O(1)	O(deg(i))	
get_neighbours	O(n)	O(1)	
in_degree_of	O(n)	O(n+m)	
out_degree_of	O(n)	O(1)	

Edge List:

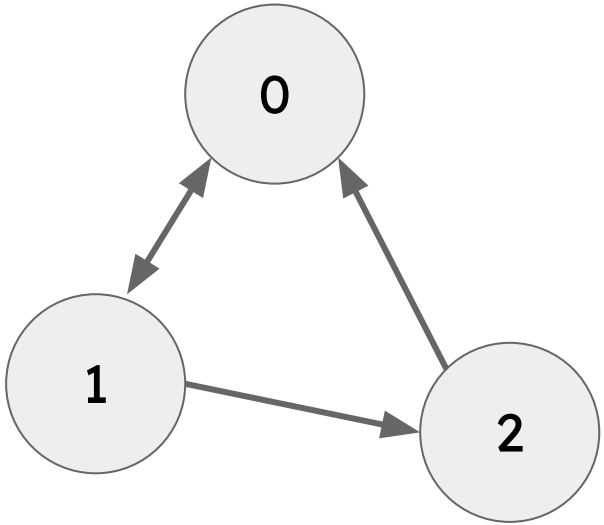
Given **n** nodes and **m** edges, we store it as an array of **m** elements.

If the graph is unweighted, then the elements are pairs (u, v) that represent the edge (u, v).

If the graph is weighted, then the elements are triplets (u, v, w) that present the edge (u, v) with weight w.

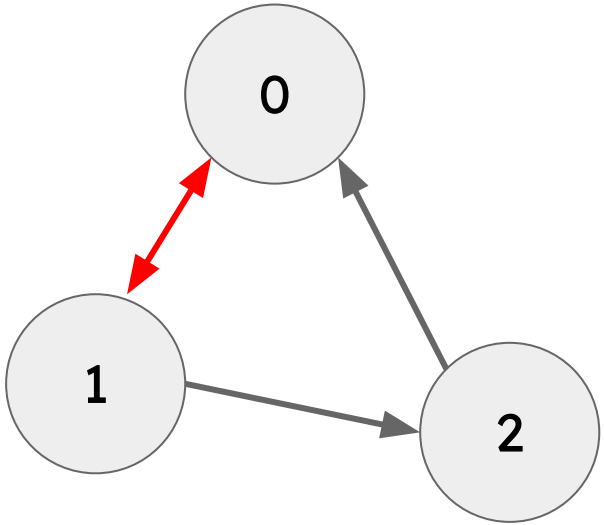
Edge List:

(0, 1)	(1, 0)	(1, 2)	(2, 0)



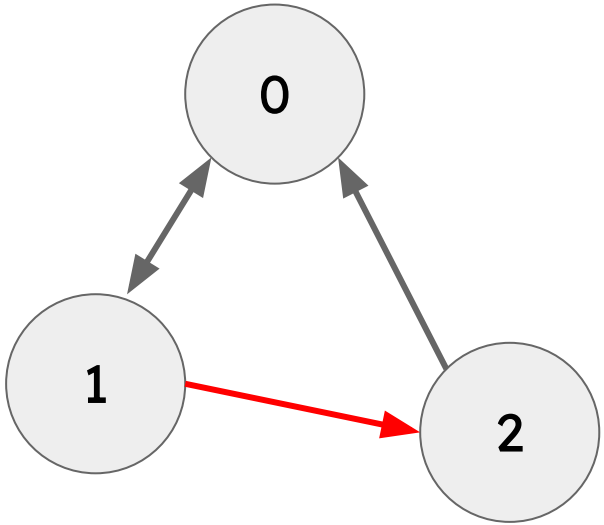
Edge List:

(0, 1)	(1, 0)	(1, 2)	(2, 0)



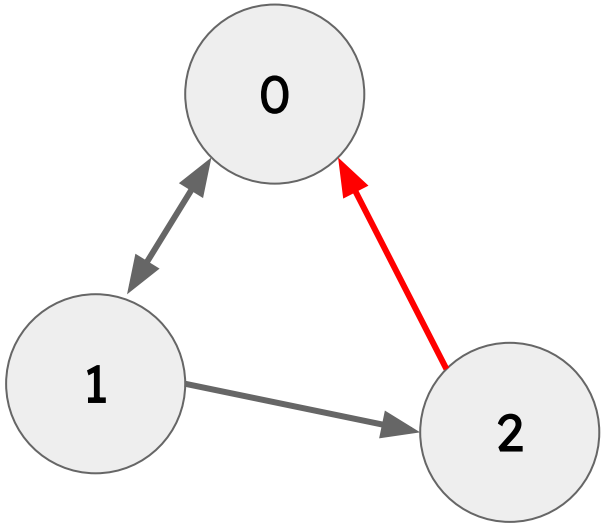
Edge List:

(0, 1)	(1, 0)	(1, 2)	(2, 0)

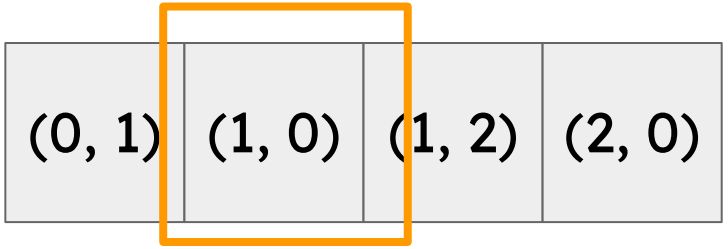


Edge List:

(0, 1)	(1, 0)	(1, 2)	(2, 0)



Edge List:

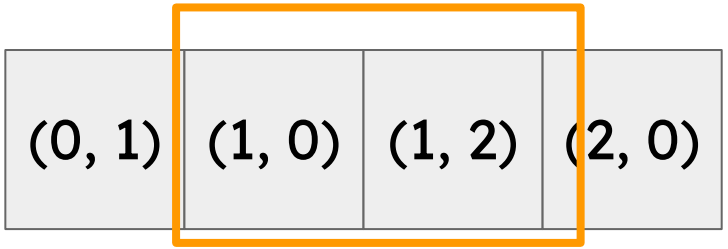


is_adjacent(1, 0)

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

Look for whether the edge (i, j) exists in the list or not.

Edge List:

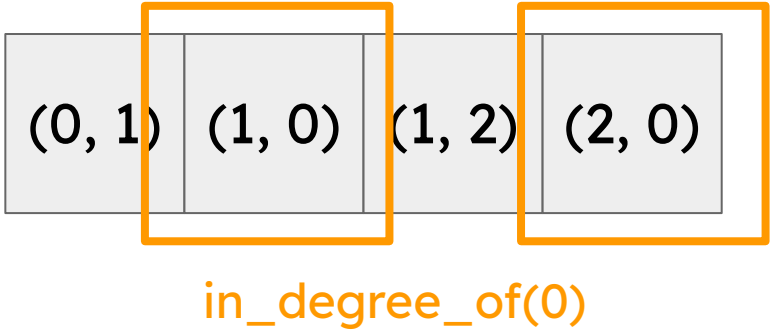


get_neighbours(1)

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

Filter the list for pairs such that the first element is i.

Edge List:



1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

Get the size of the filtered list of elements such that the second element is i.

Edge List:

1. is_adjacent(i, j)
2. get_neighbours(i)
3. in_degree_of(i)
4. out_degree_of(i)

(0, 1)	(1, 0)	(1, 2)	(2, 0)

out_degree_of(2)

Get the size of the filtered list of elements such that the first element is i.

Complexities:

	Adjacency Matrix	Adjacency List	Edge List
is_adjacent	O(1)	O(deg(i))	O(m)
get_neighbours	O(n)	O(1)	O(m)
in_degree_of	O(n)	O(n+m)	O(m)
out_degree_of	O(n)	O(1)	O(m)

What about space usage?

Remember, there are 2 parameters here, number of nodes n, number of edges m.

- An adjacency matrix stores n x n integers. $O(n^2)$ space.
- An adjacency list stores n lists, and the total size of the sub-list is m elements. O(n + m) space.
- An edge list only stores the edges. O(m) space.

Complexities:

	Adjacency Matrix	Adjacency List	Edge List
is_adjacent	O(1)	O(deg(i))	O(m)
get_neighbours	O(n)	O(1)	O(m)
in_degree_of	O(n)	O(n+m)	O(m)
out_degree_of	O(n)	O(1)	O(m)
space	$O(n^2)$	O(n+m)	O(m)

Why?

Q: Why do we learn so many ways to represent the same thing?

A: Because depending on the algorithm we implement, we might care about specific operations more than others!

	Adjacency Matrix	Adjacency List	Edge List
is_adjacent	O(1)	O(deg(i))	O(m)
get_neighbours	O(n)	O(1)	O(m)
in_degree_of	O(n)	O(n+m)	O(m)
out_degree_of	O(n)	O(1)	O(m)
space	$O(n^2)$	O(n+m)	O(m)

Can you think of one advantage the edge list might have over the other two data structures?

Pop Quiz

	Adjacency Matrix	Adjacency List	Edge List
is_adjacent	O(1)	O(deg(i))	O(m)
get_neighbours	O(n)	O(1)	O(m)
in_degree_of	O(n)	O(n+m)	O(m)
out_degree_of	O(n)	O(1)	O(m)
space	$O(n^2)$	O(n+m)	O(m)

What if we only had m < n edges? Then the edge list uses the least space!

	Adjacency Matrix	Adjacency List	Edge List
is_adjacent	O(1)	O(deg(i))	O(m)
get_neighbours	O(n)	O(1)	O(m)
in_degree_of	O(n)	O(n+m)	O(m)
out_degree_of	O(n)	O(1)	O(m)
space	$O(n^2)$	O(n+m)	O(m)

Moral of the story: depending on what you want to optimise for, what operations you use, different data structures might be preferred!

Special Shapes of Graphs

Some of the common graph structures we deal with

Honestly just a quick CS1231S refresher

Common “Shapes”:

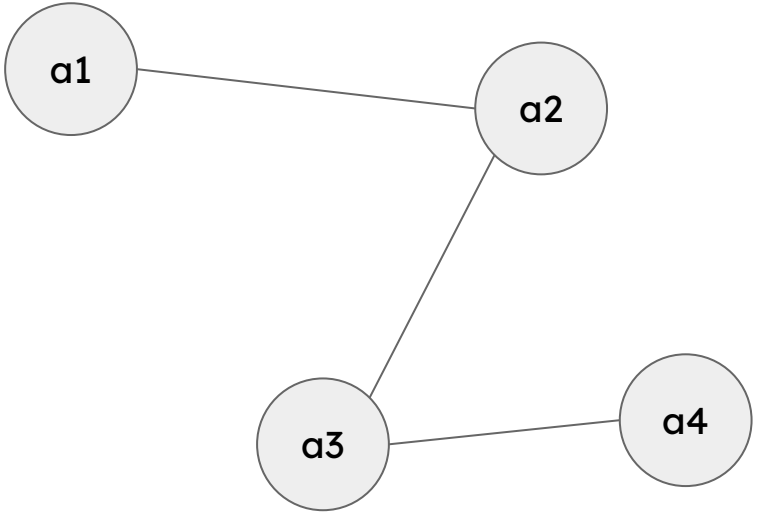
As a quick refresher from CS1231S, there are a few common shapes of graphs we care about.

1. Simple paths
2. Trees
3. Cycles
4. DAGs
5. Complete graphs

Simple path:

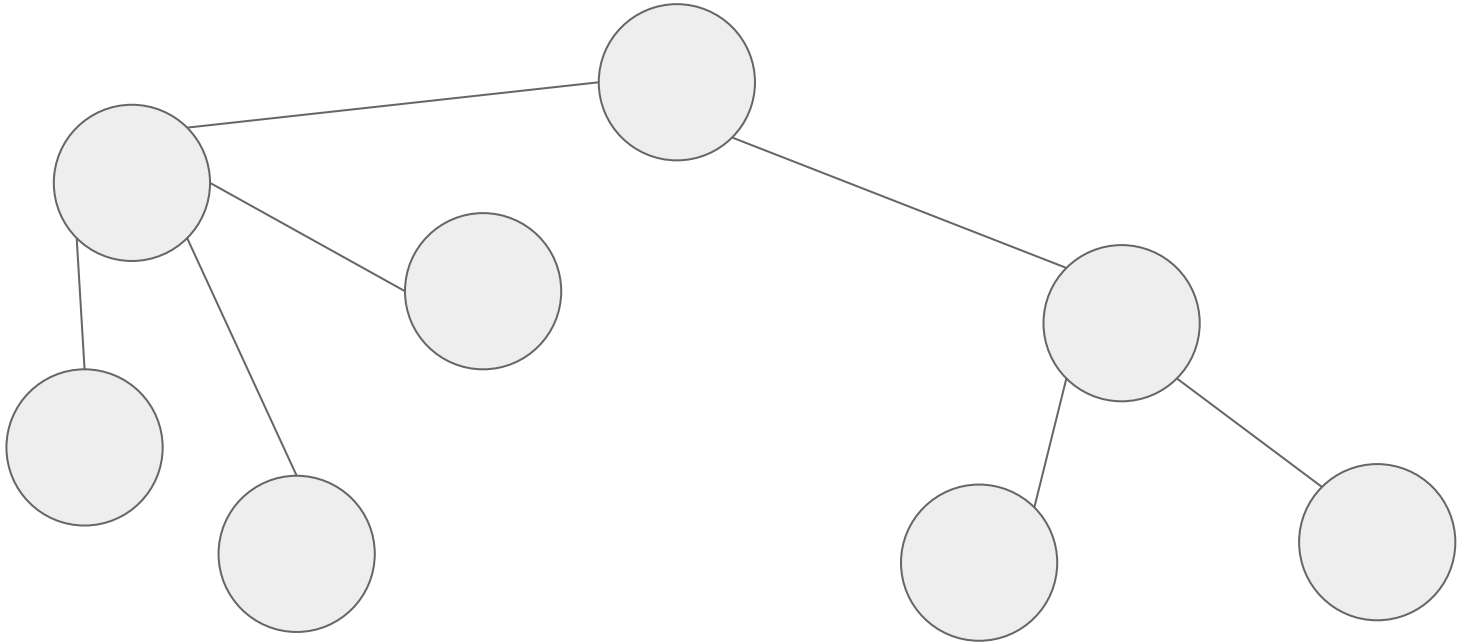
A simple path from node u to node v is a sequence of nodes a_1 a_2 a_3 … a_s such that no nodes are repeated and:

1. $a_1 = u$
2. $a_s = v$
3. a_i has an edge to a_{i+1}



Tree:

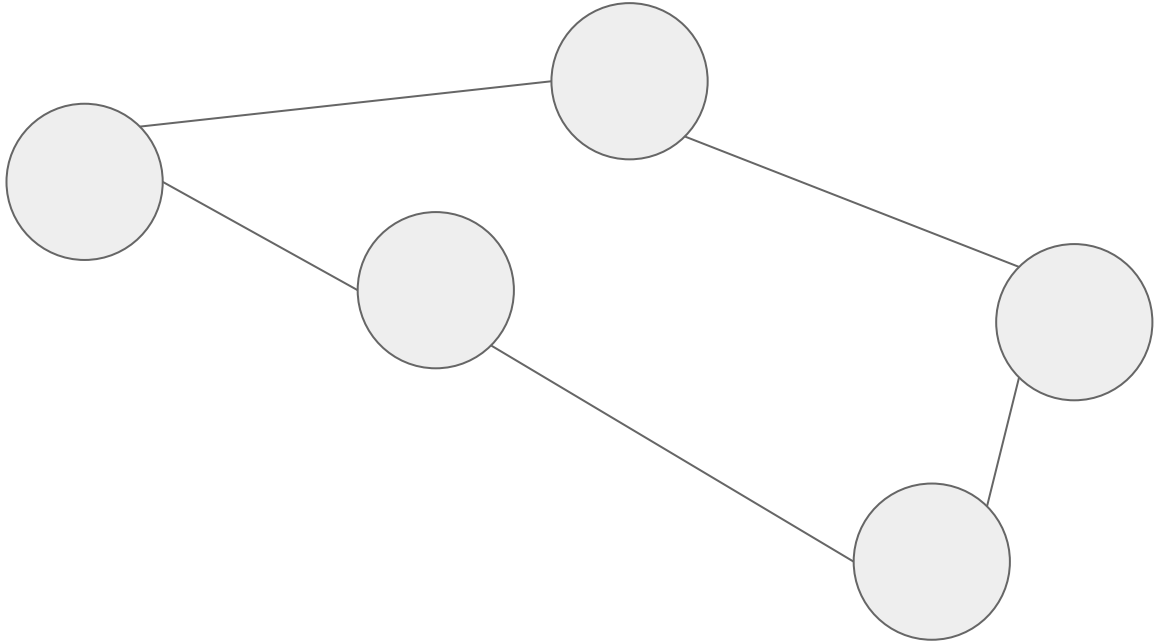
A tree is a connected graph that is cycle-free.



Cycle:

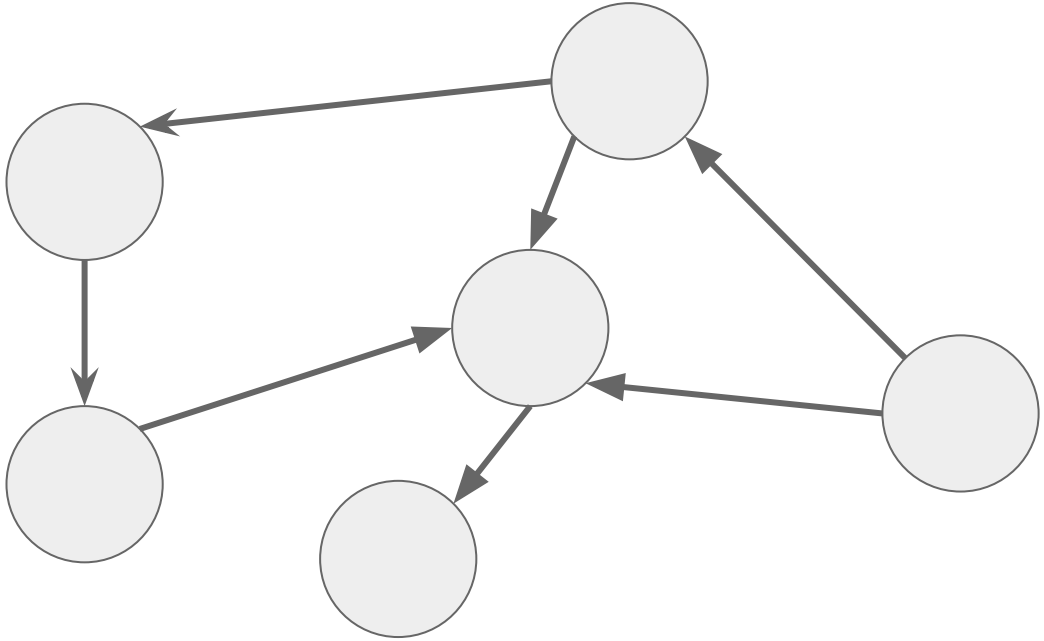
A cycle of a graph G is a sub-graph such that it is a sequence of nodes a_1 a_2 a_3 … a_s such that no nodes are repeated and:

1. $s >= 3$
2. $a_1 = a_s$
3. a_i has an edge to a_{i+1}



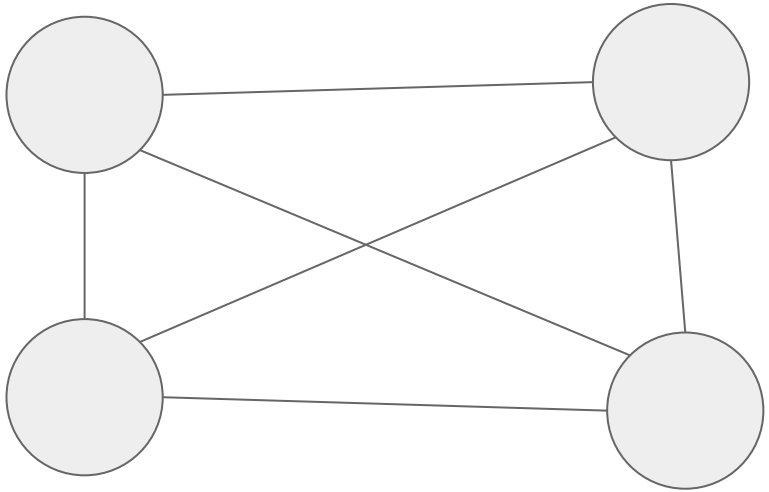
DAG:

A DAG is a directed, acyclic graph.



Complete Graph:

A complete graph is when all n nodes have edges to any other node.



Graph Problems In General

Traversing, Shortest Pathfinding, etc

Graph traversal is the hammer, and everything else is a nail

Graph Problems in General:

Now that we're in the second half of the semester, it might look a little weird to narrow ourselves down to only graph problems.

Turns out that there are a lot of graph problems. Even seemingly unrelated problems can often be converted into one.

Graph Problems Technique:

A good technique is to learn from the myriad of algorithms we're going to show you. Then either:

1. Modify an existing algorithm to solve a new problem.

2. Black box the a known algorithm, but transform our problem into one our algorithm will solve.

We'll do some of this today to show you what we mean.

DFS

Depth First Search

Not, not the duty free store in Changi…

Depth First Search:

This is probably the first algorithm that everyone learns on graph traversal.

It’s typically less of an algorithm that is useful by itself, and more useful as a “template” to modify to get stuff done.

Nonetheless, let’s look at DFS.

Graph Traversal Problem:

Input: A graph G = (V, E) and a source node s.
(Not necessarily connected, not necessarily undirected.)

Output: A list of nodes that can be visited from node s.

Ver 1.0

dfs_get_visited_list(G, s, result_list):

Get the neighbour list of s

Iterate through all neighbours n of s (via G.get_neighbours(s))

Recursively compute which nodes n can reach

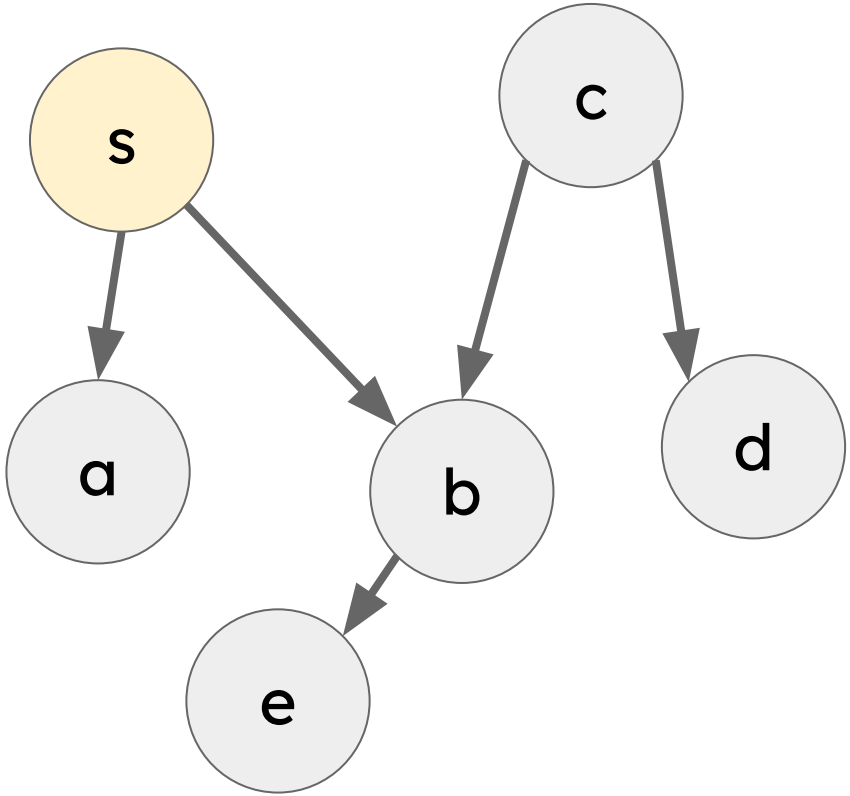
(Via call to dfs_get_visited_list(G, n, result_list))

Append s to the result_list

Ver 1.0

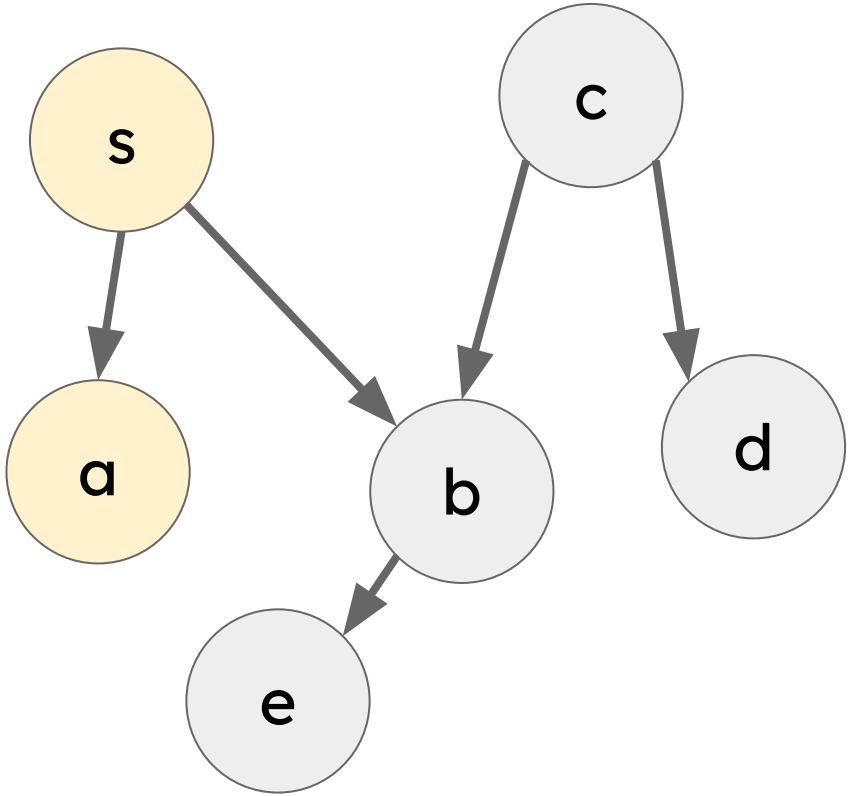
```
dfs_get_visited_list(G, s, result_list):


    neighbour_list = G.neighbours_of(s)

    for neighbour n in neighbour_list:

        dfs_get_visited_list(G, n, result_list)

    result_list.append(s)
```

Ver 1.0



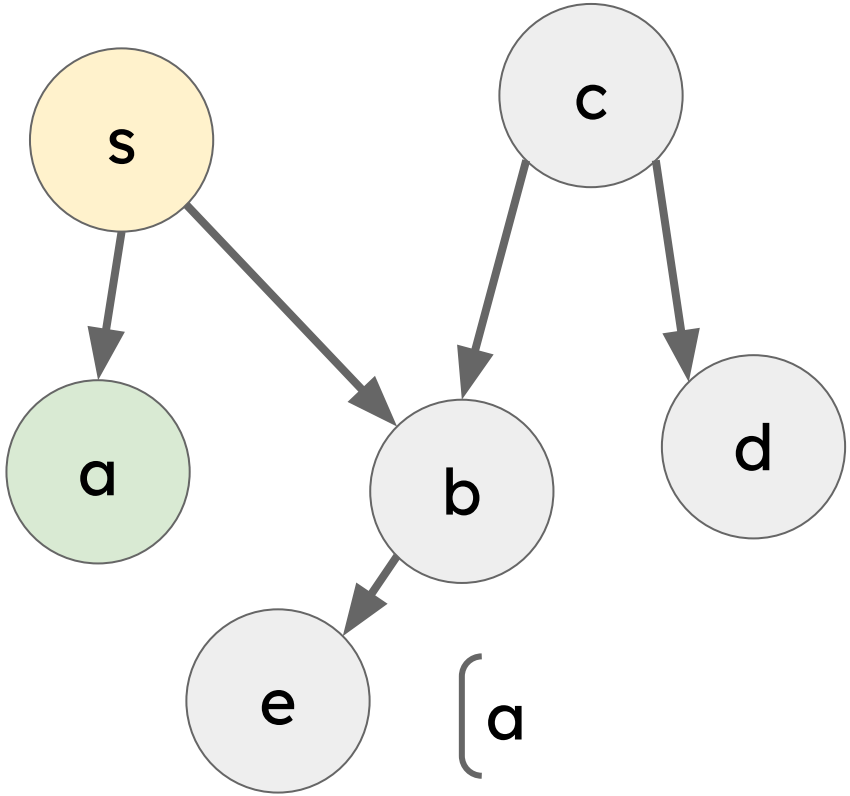
Start from node s, we visit the neighbours of s.

Ver 1.0



First neighbour a.

Ver 1.0

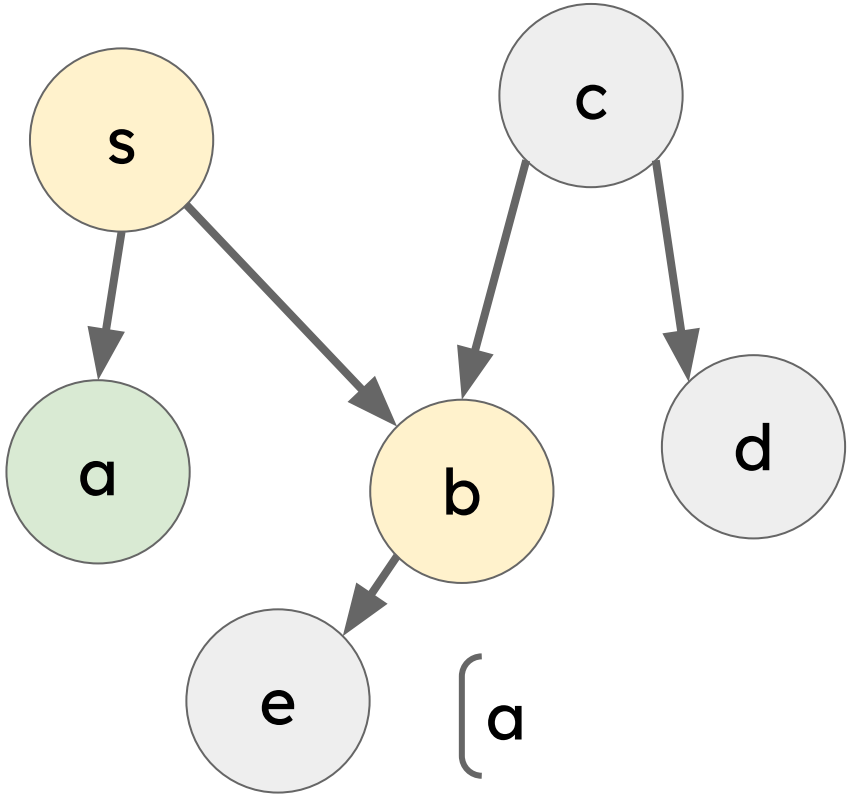


First neighbour a.

a has no more neighbours, append itself to the list. Then return

[a]

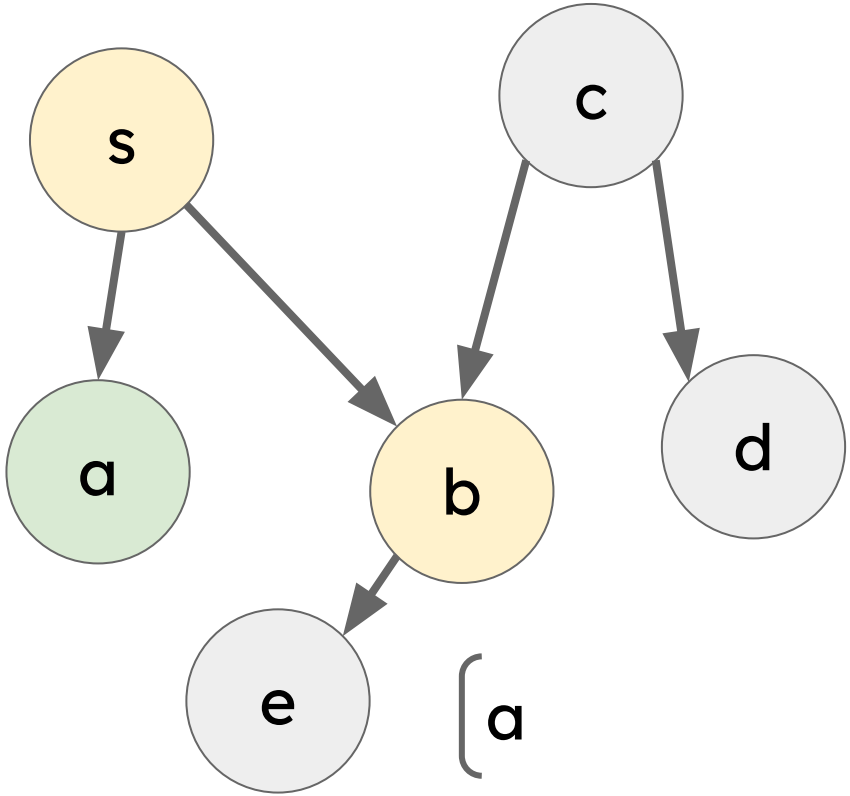
Ver 1.0



Next, neighbour b.

[a]

Ver 1.0

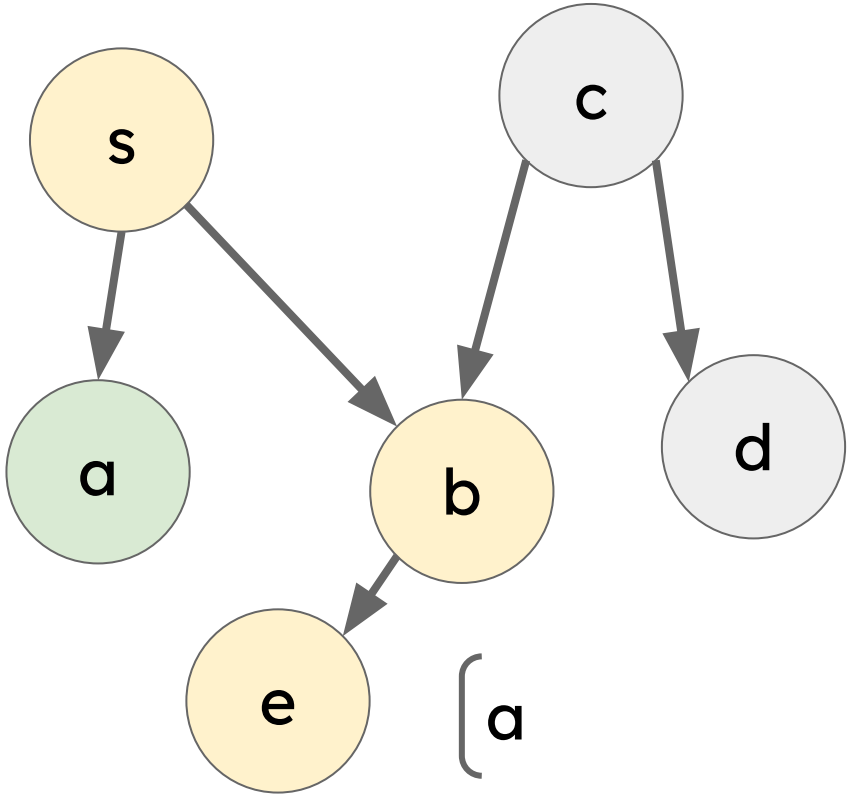


Next, neighbour b.

Neighbour b also has a neighbour.
Recurse on that.

[a]

Ver 1.0



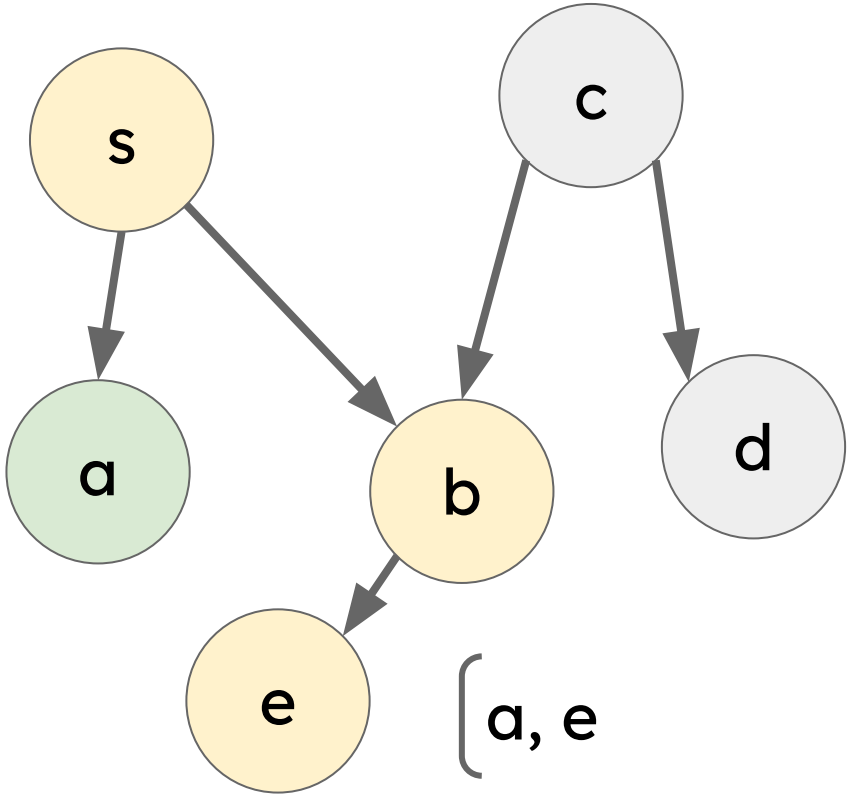
Next, neighbour b.

Neighbour b also has a neighbour.
Recurse on that.

Visit e

[a]

Ver 1.0



[a, e]

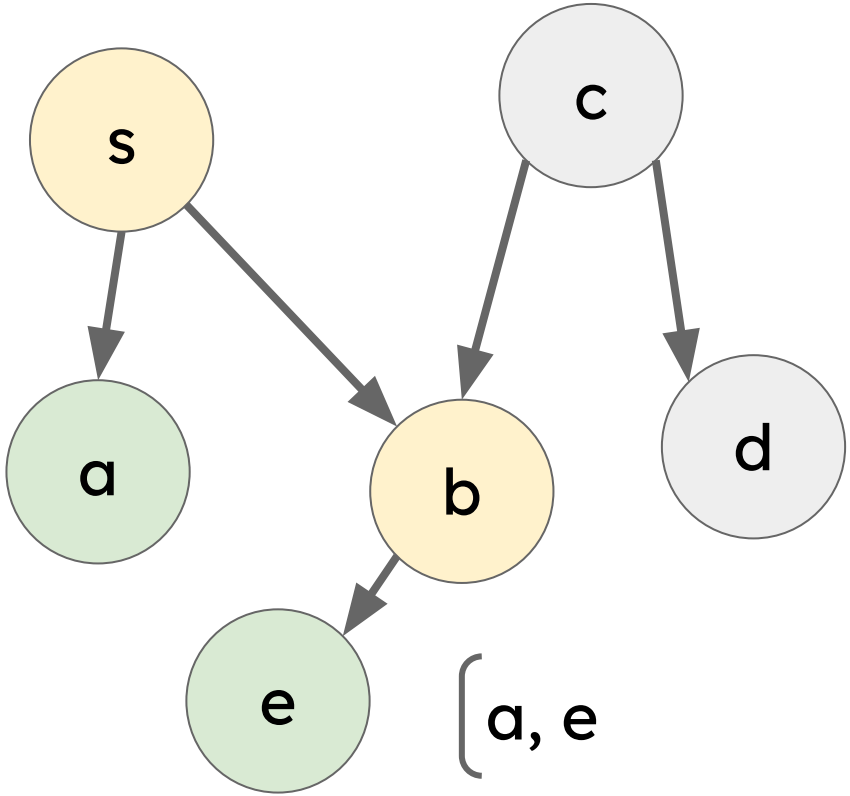
Next, neighbour b.

Neighbour b also has a neighbour.
Recurse on that.

Visit e, no other neighbours.

Append e.

Ver 1.0



[a, e]

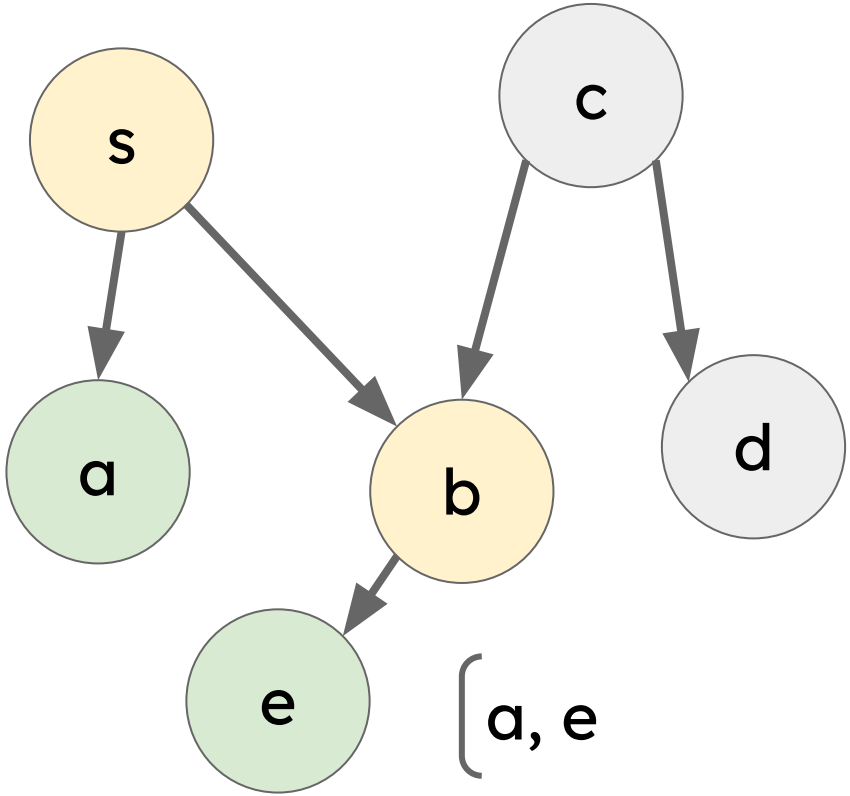
Next, neighbour b.

Neighbour b also has a neighbour.
Recurse on that.

Visit e, no other neighbours.

Append e.

Ver 1.0



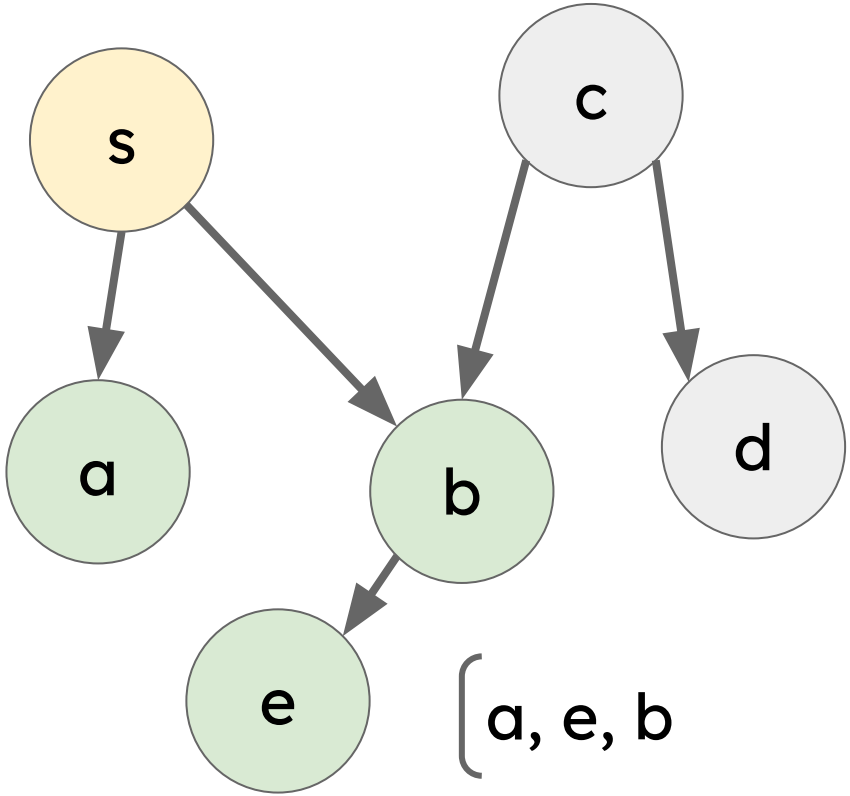
Next, neighbour b.

Neighbour b has no other neighbour.

Append b.

[a, e]

Ver 1.0



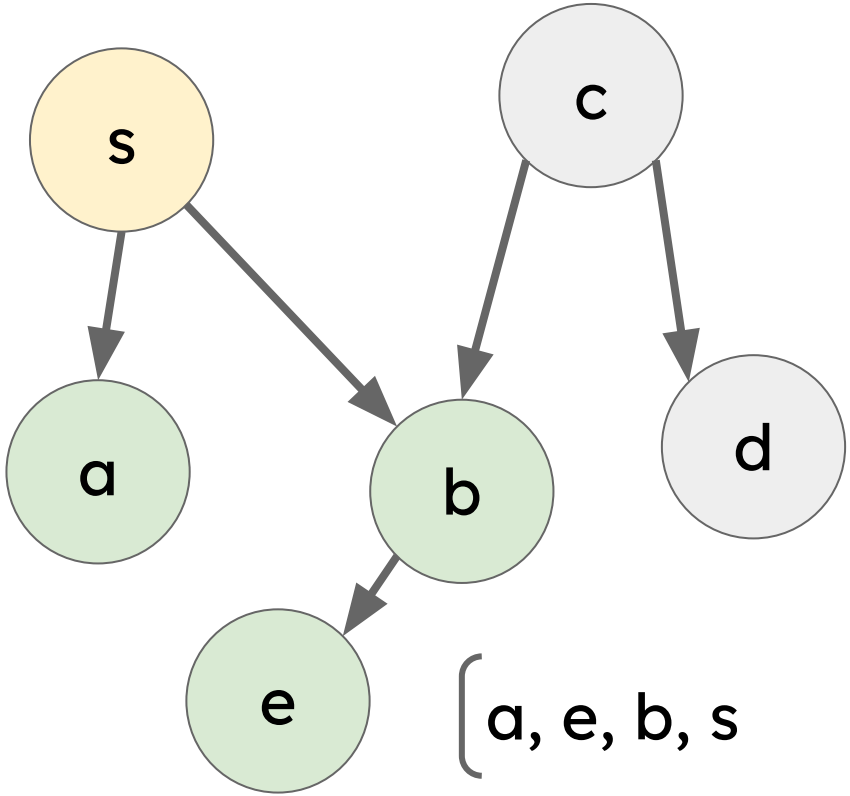
Next, neighbour b.

Neighbour b has no other neighbour.

Append b.

[a, e, b]

Ver 1.0



Done with s, append s.

[a, e, b, s]

dfs_get_visited_list(G, s, result_list):

Get the neighbour list of s

Iterate through all neighbours n of s (via G.get_neighbours(s))

Recursively compute which nodes n can reach

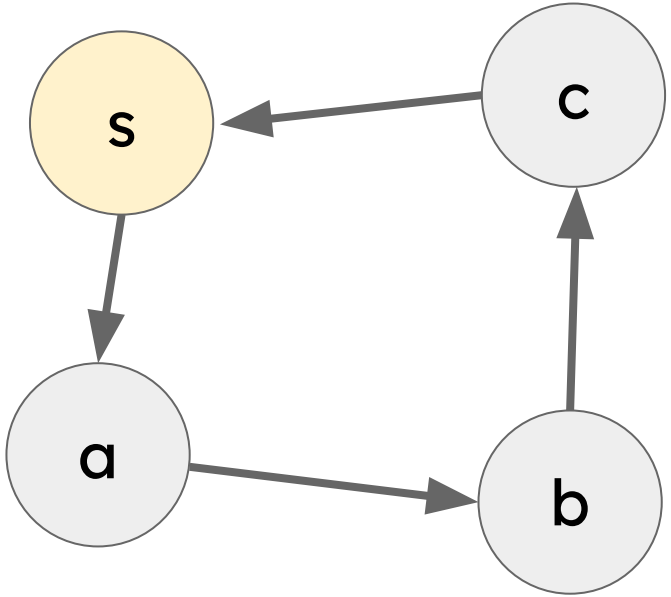
(Via call to dfs_get_visited_list(G, n, result_list))

Append s to the result_list

- Does it work?
- What's the runtime?

Pop Quiz

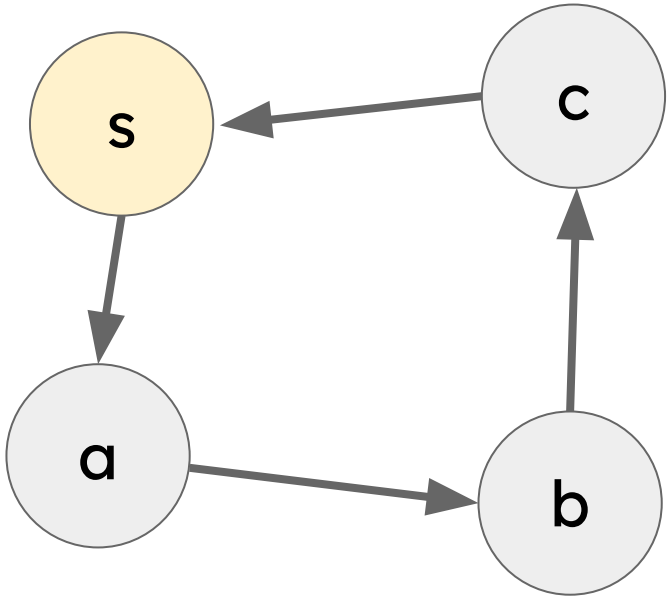
Ver 1.0



What about this graph?

The algorithm won’t terminate!

Ver 2.0



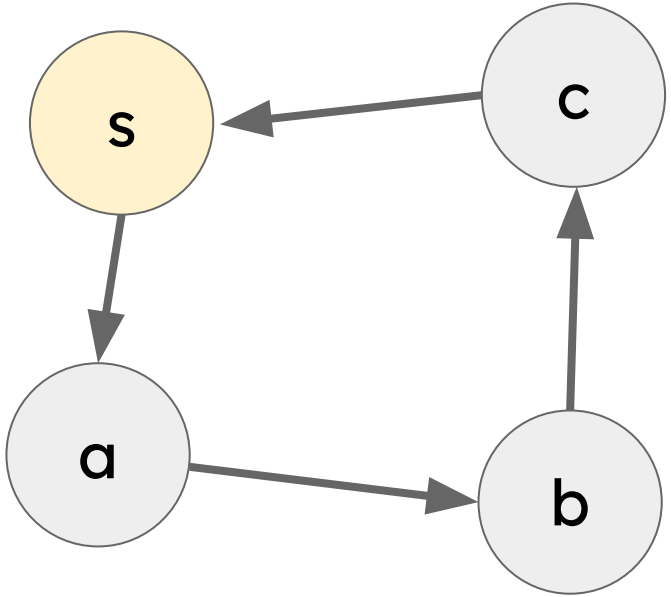
We need a data structure to mark the vertices we've seen before.

How fast can we do this?

1. O(1) worst-case
2. O(1) expected
3. O(log n) worst-case
4. O(n) worst-case

Pop Quiz

Ver 2.0



Design considerations:

The number of nodes are known in advance, and if we treat them as a running numbers from 0 to n - 1, we only need to use an array to mark whether it is visited or not.

Furthermore, since we have a visited list, we can just use that itself to figure out which nodes have been visited.

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

```
    If s has been visited before, just return
    Mark s as visited
    Get the neighbour list of s
    Iterate through all neighbours n of s
        Recurse on n
```

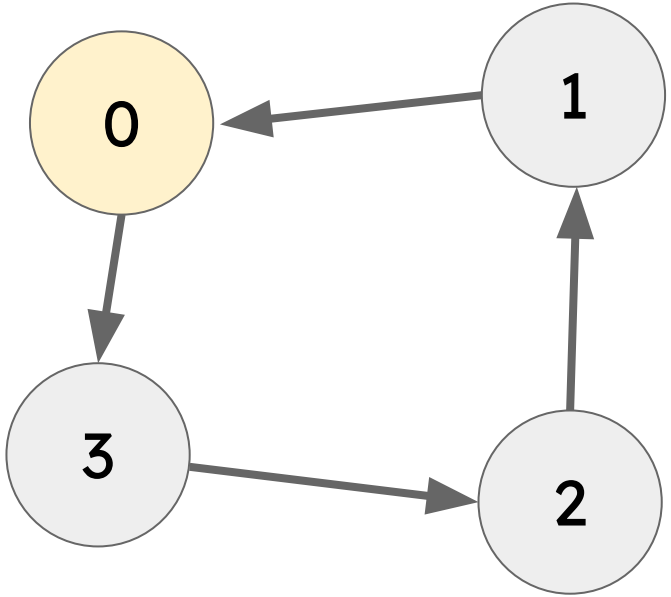
After we run this, just output the indices v on visited_list for which visited_list[v] = true

Ver 2.0

```
dfs_get_visited_list(G, s, visited_list):
    if visited_list[s] == true: return;
    visited_list[s] = true
    neighbour_list = G.neighbours_of(s)
    for neighbour n in neighbour_list :
        dfs_get_visited_list(G, n, visited_list)
```

After we run this, just output the indices v on visited_list for which visited_list[v] = true

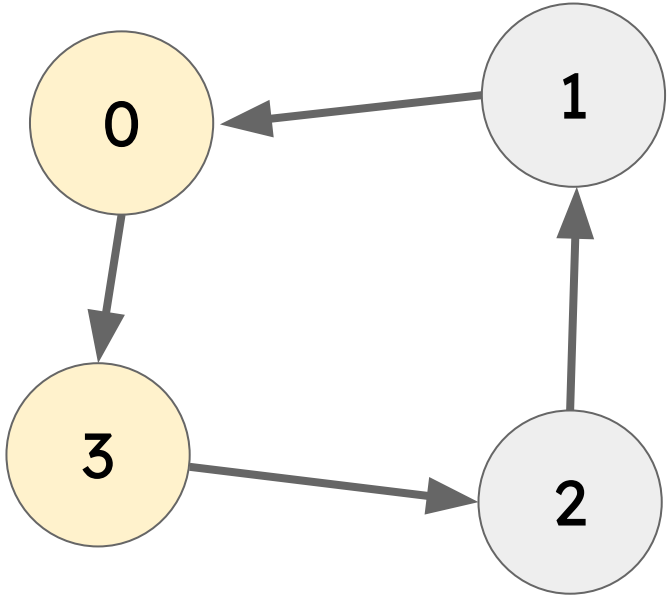
Ver 2.0



Starting on node 0, we see that it is not visited, so we visit it and all its neighbours.

(true, false, false, false)

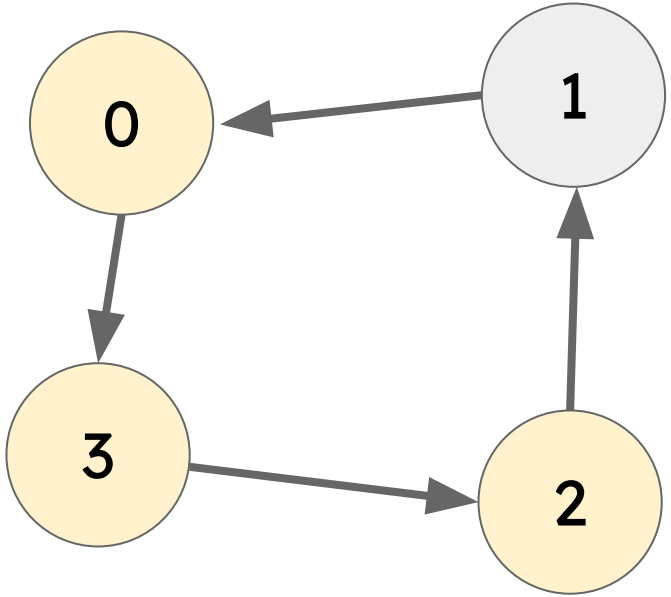
Ver 2.0



Starting on node 0, we see that it is not visited, so we visit it and all its neighbours.

[true, false, false, true]

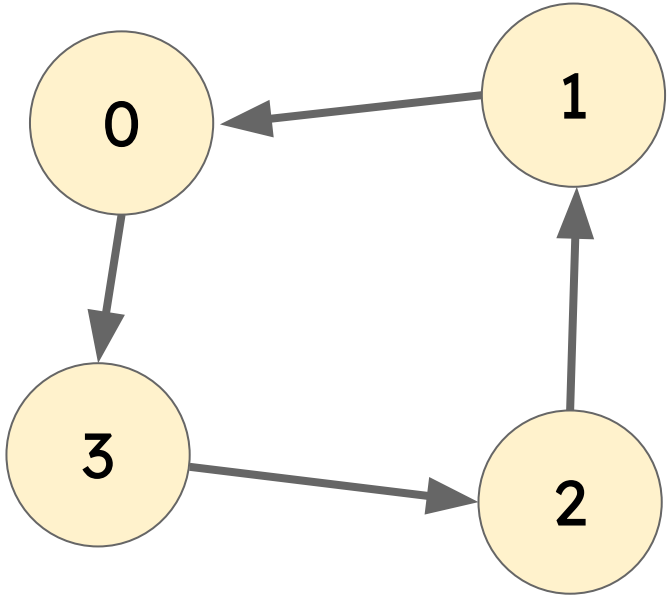
Ver 2.0



Starting on node 0, we see that it is not visited, so we visit it and all its neighbours.

[true, false, true, true]

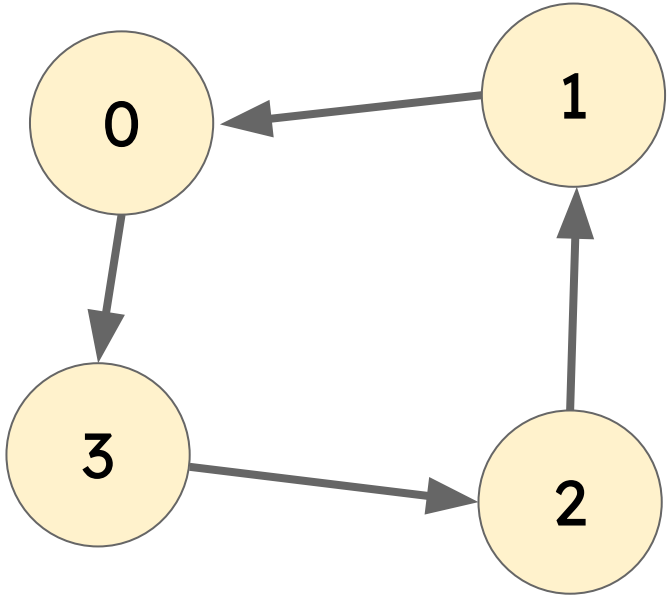
Ver 2.0



Starting on node 0, we see that it is not visited, so we visit it and all its neighbours.

[true, true, true, true]

Ver 2.0

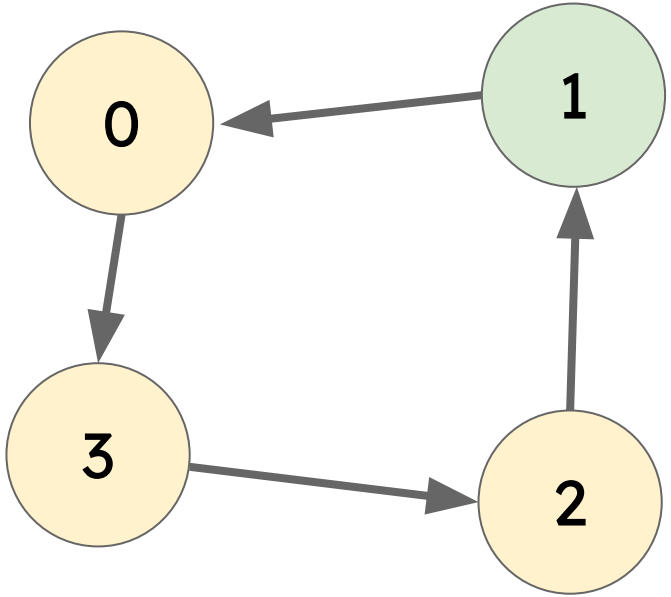


[true, true, true, true]

At node 1, we still check its neighbours and realise that node 0 has already been marked as visited.

So we do not visit it. And instead we return.

Ver 2.0

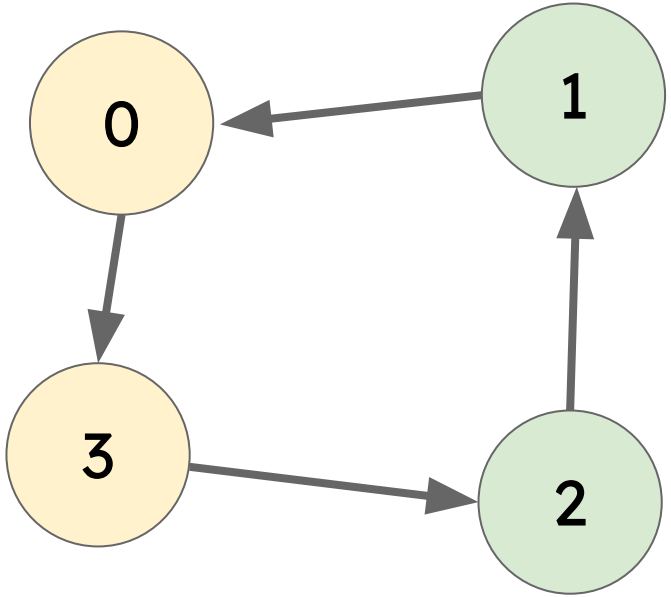


[true, true, true, true]

At node 1, we still check its neighbours and realise that node 0 has already been marked as visited.

So we do not visit it. And instead we return.

Ver 2.0

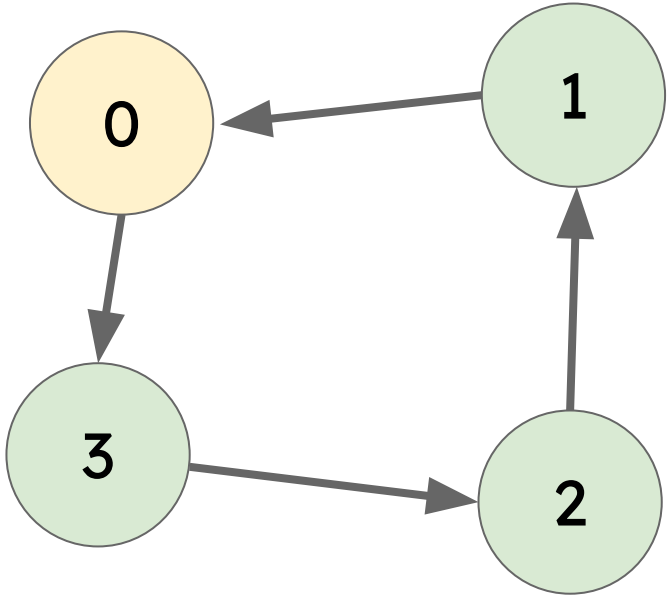


[true, true, true, true]

At node 1, we still check its neighbours and realise that node 0 has already been marked as visited.

So we do not visit it. And instead we return.

Ver 2.0

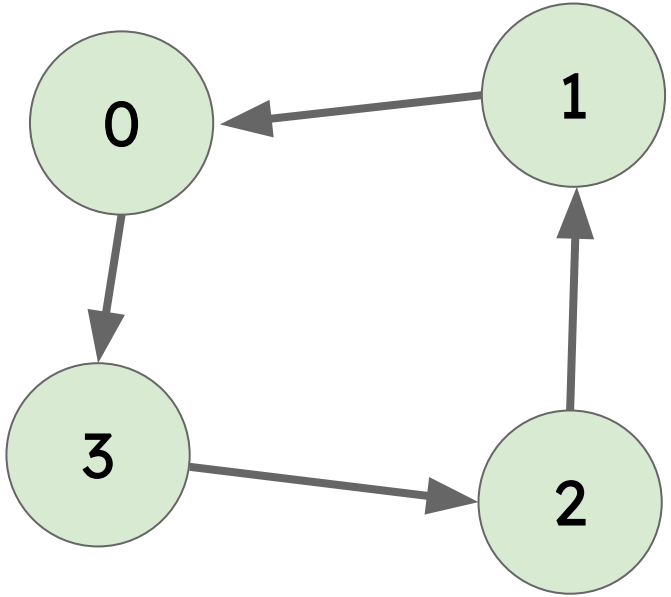


[true, true, true, true]

At node 1, we still check its neighbours and realise that node 0 has already been marked as visited.

So we do not visit it. And instead we return.

Ver 2.0



[true, true, true, true]

At node 1, we still check its neighbours and realise that node 0 has already been marked as visited.

So we do not visit it. And instead we return.

```
dfs_get_visited_list(G, s, visited_list):
    If s has been visited before, just return
    Mark s as visited
    Get the neighbour list of s
    Iterate through all neighbours n of s
        Recurse on n
```

- Does it work?
- What's the runtime?

Pop Quiz

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as been visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

Observations:

1. We only visit each node at most once.
2. For every node v, we iterate for at most deg(v) iterations.

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

How much does it cost to check and mark a node as visited?

1. O(1)
2. O(n)
3. O(m)

Pop Quiz

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

How much does it cost to get the neighbour list of s?

1. O(1)
2. O(n)
3. O(m)

Pop Quiz

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

It depends! What data structure are we using?

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

How much does it cost to check and mark a node as visited?

1. O(1)
2. O(n)
3. O(m)

If using an array.

Pop Quiz

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

How much does it cost to get the neighbour list of s?

1. O(1)
2. O(n)
3. O(m)

If using an adjacency list

Pop Quiz

$\deg(v_0) + \deg(v_1) + \deg(v_2) + \ldots + \deg(v_{n-1}) = ???$

1. $O(n)$
2. ✓ $O(m)$
3. $O(n^2)$
4. $O(2^n)$

Theorem 10.1.1 The Handshake Theorem

If G is any graph, then the sum of the degrees of all the vertices of G equals twice the number of edges of G. Specifically, if the vertices of G are $v_1, v_2, \ldots, v_n$, where $n \geq 0$, then

$$\text{The total degree of } G = \deg(v_1) + \deg(v_2) + \ldots + \deg(v_n)$$
$$= 2 \times (\text{the number of edges of } G).$$

Pop Quiz

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

So if we're starting from some fixed node s, in total O(m) time.

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

Except! That's a **lie**.

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

If s has been visited before, just return

Mark s as visited

Get the neighbour list of s

Iterate through all neighbours n of s

Recurse on n

Hint: there's a data structure being used here that has potentially has size > m.

Why is the runtime not O(m)?

Pop Quiz

Ver 2.0

dfs_get_visited_list(G, s, visited_list):

```
    If s has been visited before, just return

    Mark s as visited

    Get the neighbour list of s

    Iterate through all neighbours n of s

        Recurse on n
```

The visited_list array costs O(n) time to make! If n > m, then the runtime is O(n + m).

Recall from tutorial 2 problem 2 that O(n + m) can intuitively be thought of as O(max(n, m)) as well.

Ver 2.0

```
dfs_get_visited_list(G, s, visited_list):
    If s has been visited before, just return
    Mark s as visited
    Get the neighbour list of s
    Iterate through all neighbours n of s
        Recurse on n
```

We will see how to modify DFS to solve more problems later, and on the next lecture.

BFS

Breadth First Search

Sometimes I think of it as “breakfast search” or “bread-first search” cause of how it sounds.

Breadth First Search:

Notice that depth-first search tends to keep exploring a path until it can't continue, then before "backtracking" up its path to find another path to explore. So it goes for "depth" first.

Breadth-first takes the other approach where it explores path based on "breadth".

Breadth First Search:

In particular, we want to visit all the nodes of distance 0, then all nodes of distance 1, then all nodes of distance 2, then all nodes of distance 3, and so on… (this will be useful for a different problem…)

Note: DFS can naturally be written recursively, but BFS is better done **iteratively**.

Since we're doing it iteratively, let's bring up invariants again.

Traversal Template:

```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all false>
    [some data structure to store nodes to visit]
    [loop condition]:
        get some new node c_node out of data structure
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

Traversal Template:

```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all f
    [some data structure to store nodes to visit]
    [loop condition]:
        get some new node c_node out of data structure
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

As long as we have nodes to visit, we should keep going.

Traversal Template:

```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all f
    [some data structure to store nodes to visit]
    data structure is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

As long as we have nodes to visit, we should keep going.

Traversal Template:

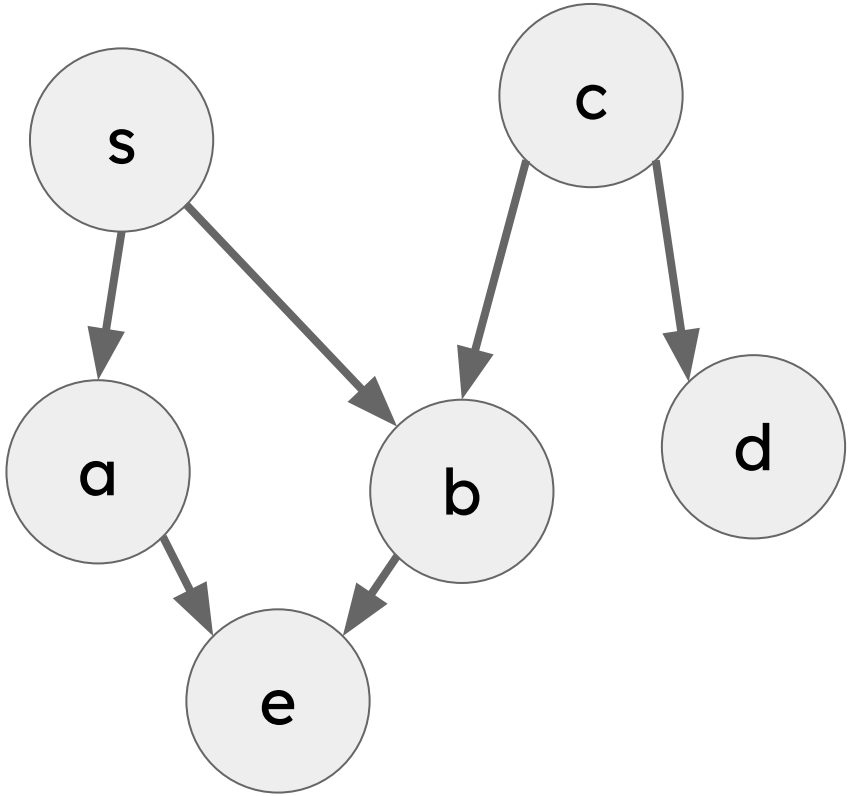
```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all f
    [some data structure to store nodes to visit]
    data structure is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

We want to choose the right data structure here to help us dictate the ordering we visit the nodes

BFS:

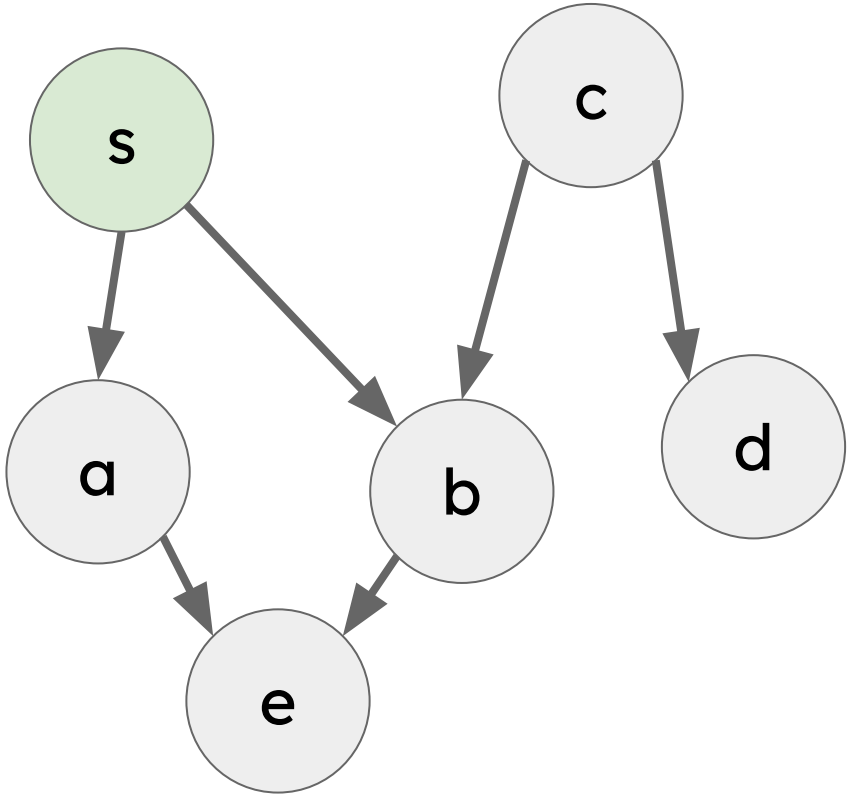
```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all false>
    queue = <a FIFO queue that holds only s>
    queue is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

BFS: [s]



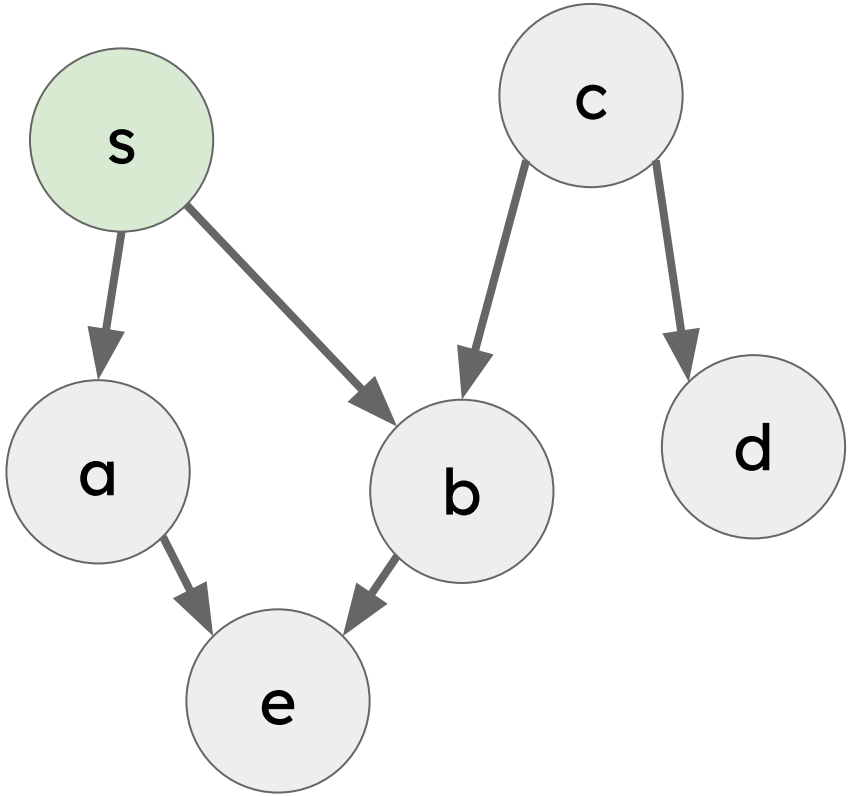
Initialise queue to only hold node s.

BFS: [ ]



Remove the next element from the queue: s. Mark as visited.

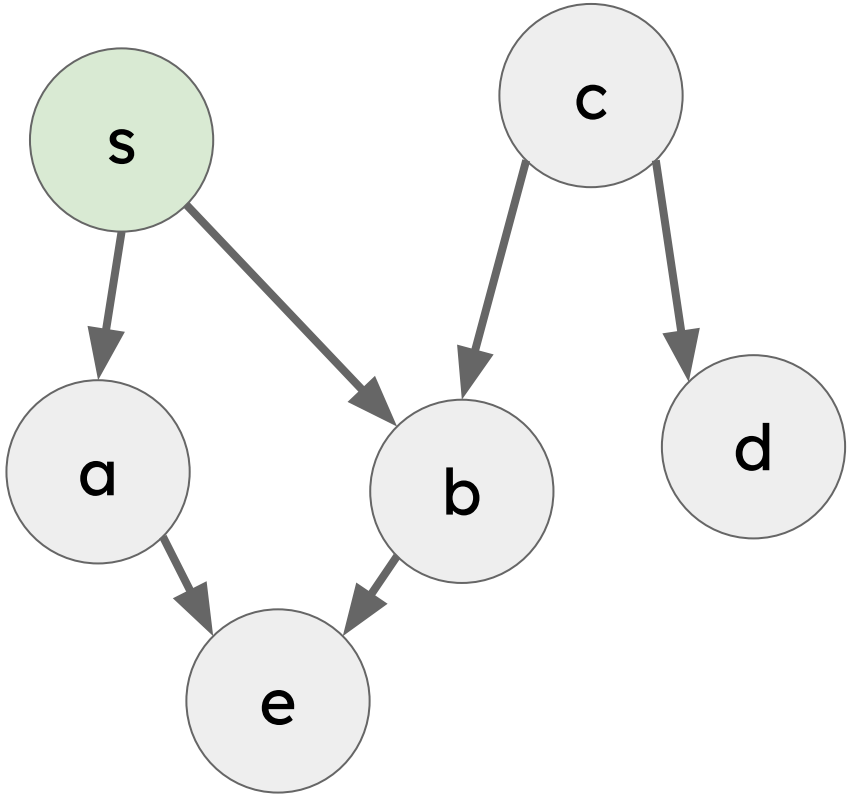
BFS: [a, b]



Add in its neighbours a, b.

Let's say we added a, then b.

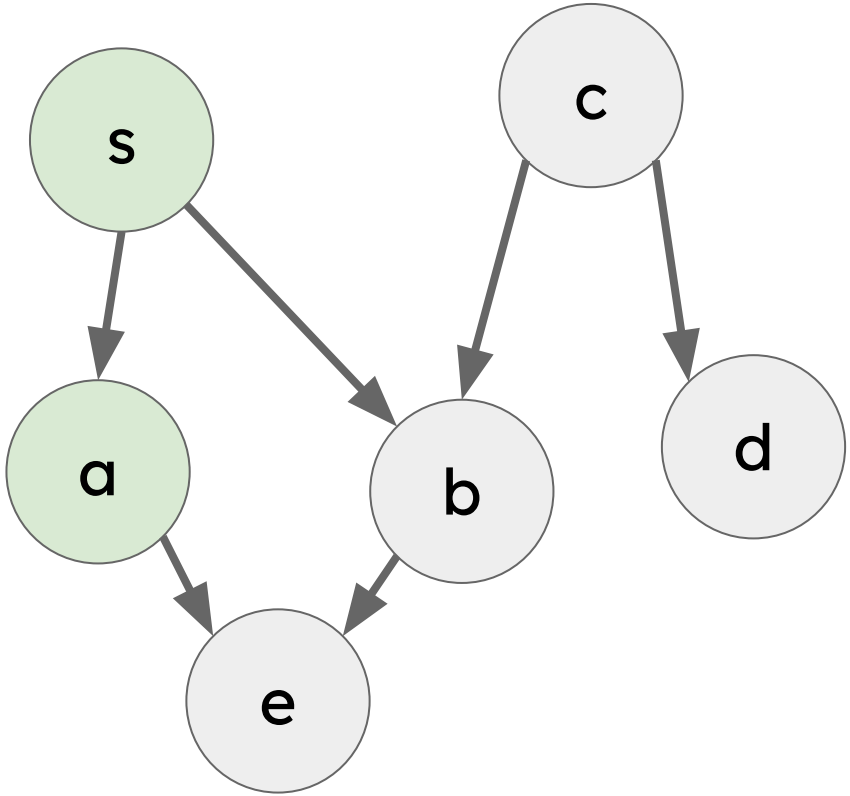
BFS: [a, b]



Since a queue is FIFO, we will take node a out first.

Mark it as visited.

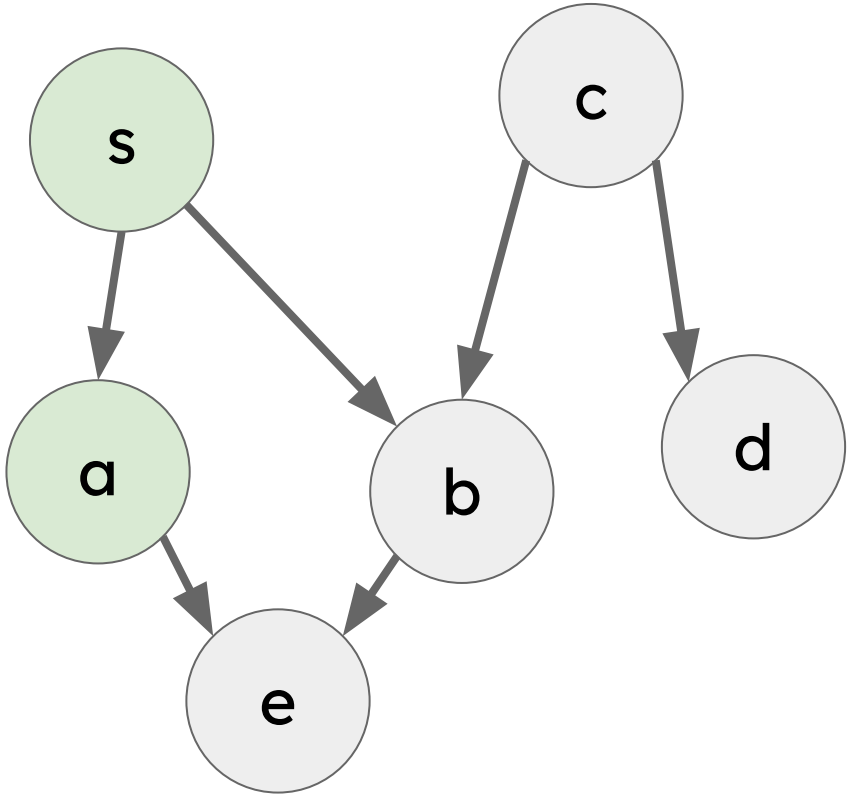
BFS: [b]



Since a queue is FIFO, we will take node a out first.

Mark it as visited. Now add its neighbours.

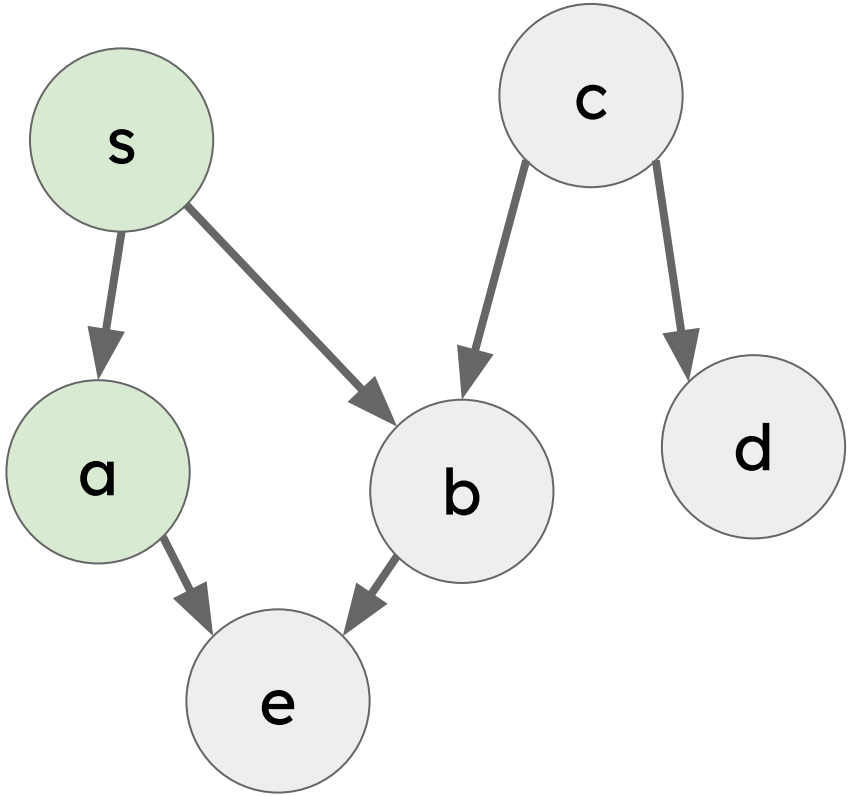
BFS: [b, e]



Since a queue is FIFO, we will take node a out first.

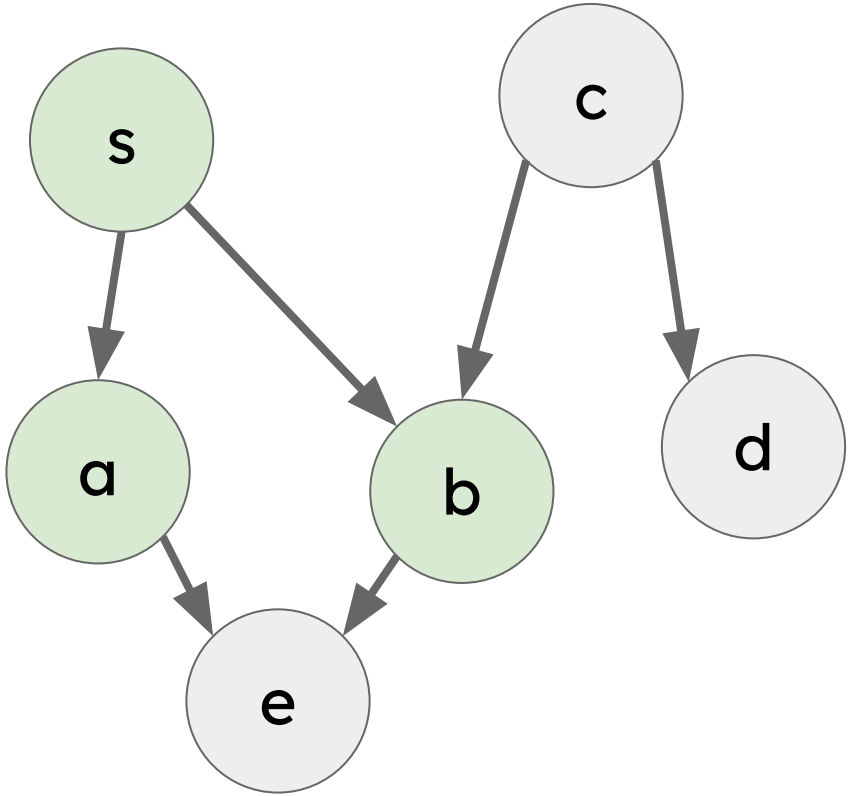
Mark it as visited. Now add its neighbours.

BFS: [b, e]



Now remove the next item from the queue: b.

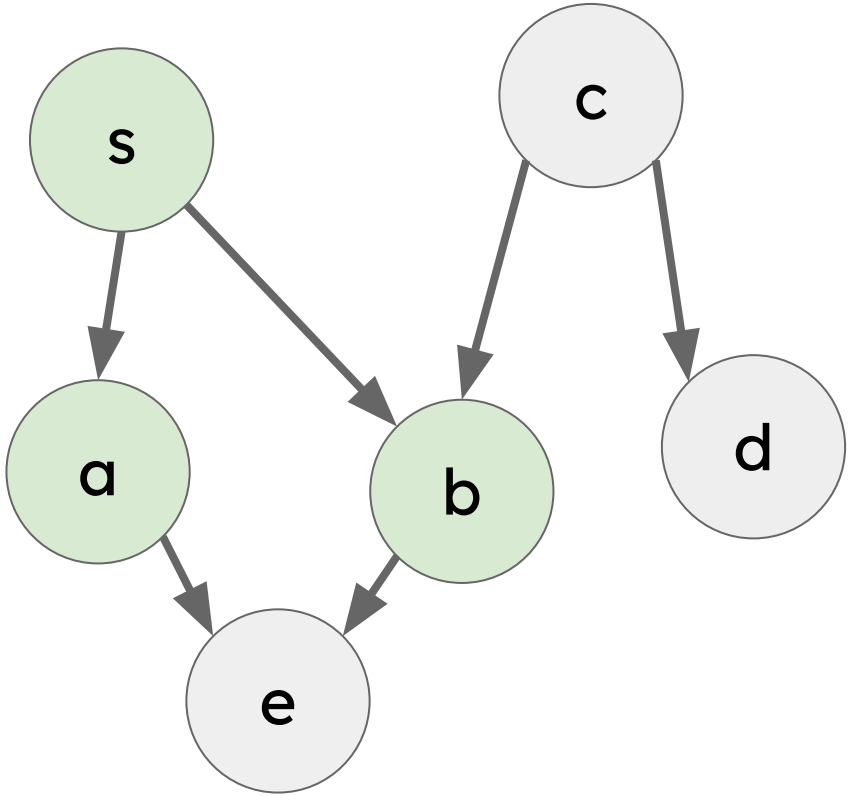
BFS: [e, e]



Now remove the next item from the queue: b.

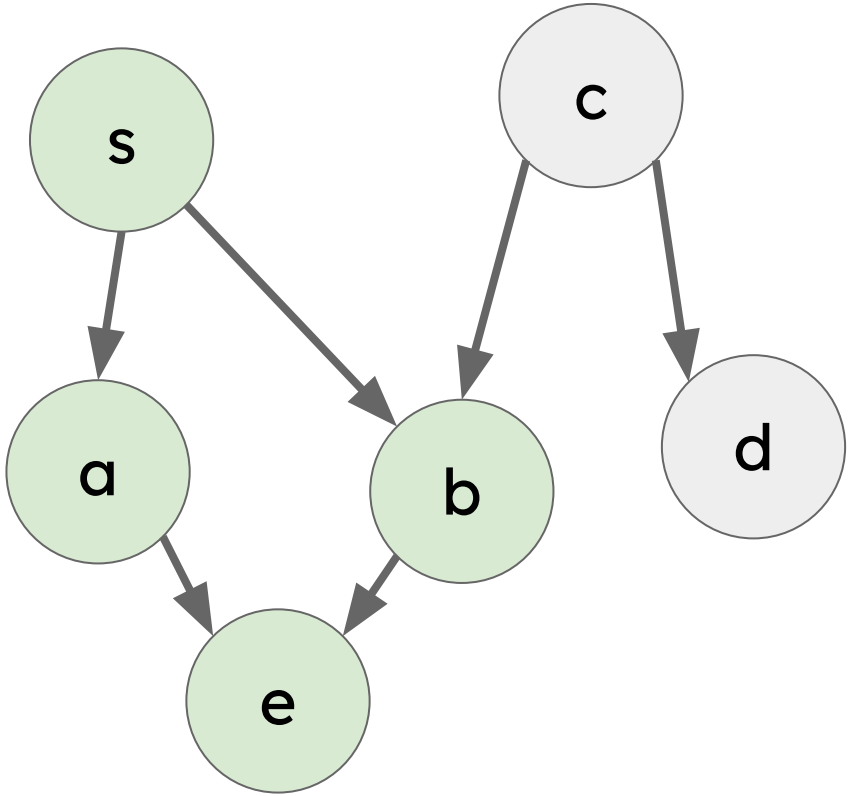
Mark as visited, add in neighbours.

BFS: [e]



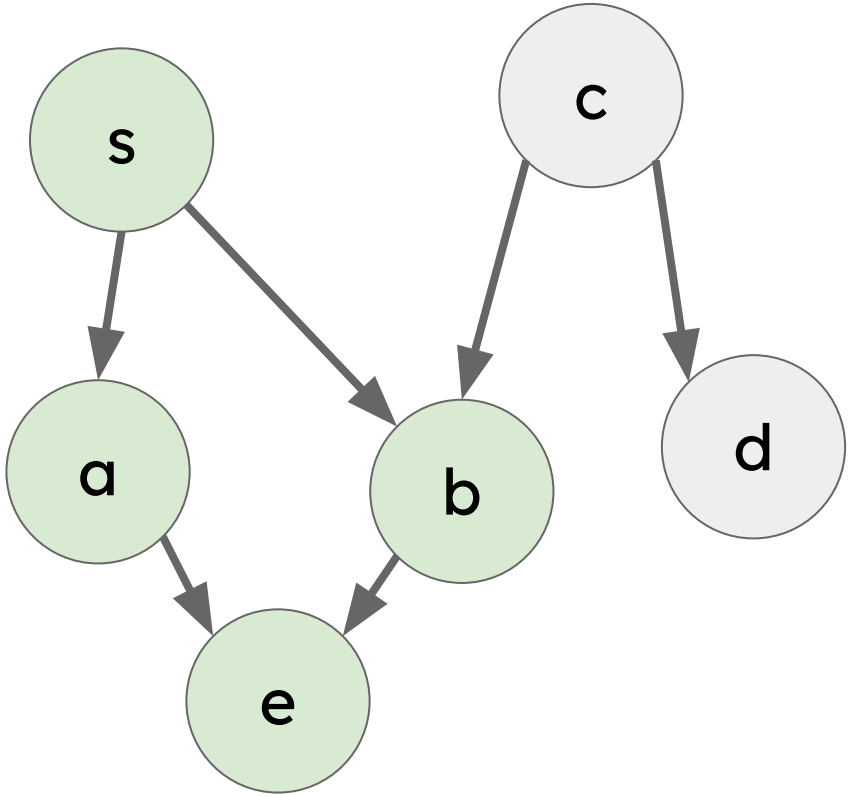
Remove the next item, e.

BFS: [e]



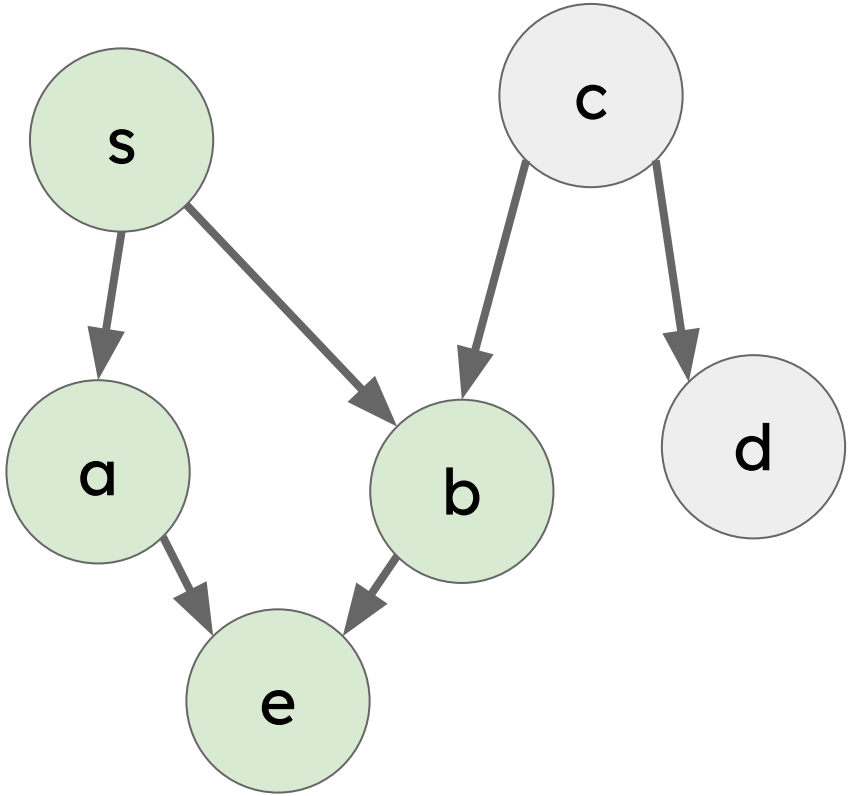
Remove the next item, e. Mark as visited.

BFS: [ ]



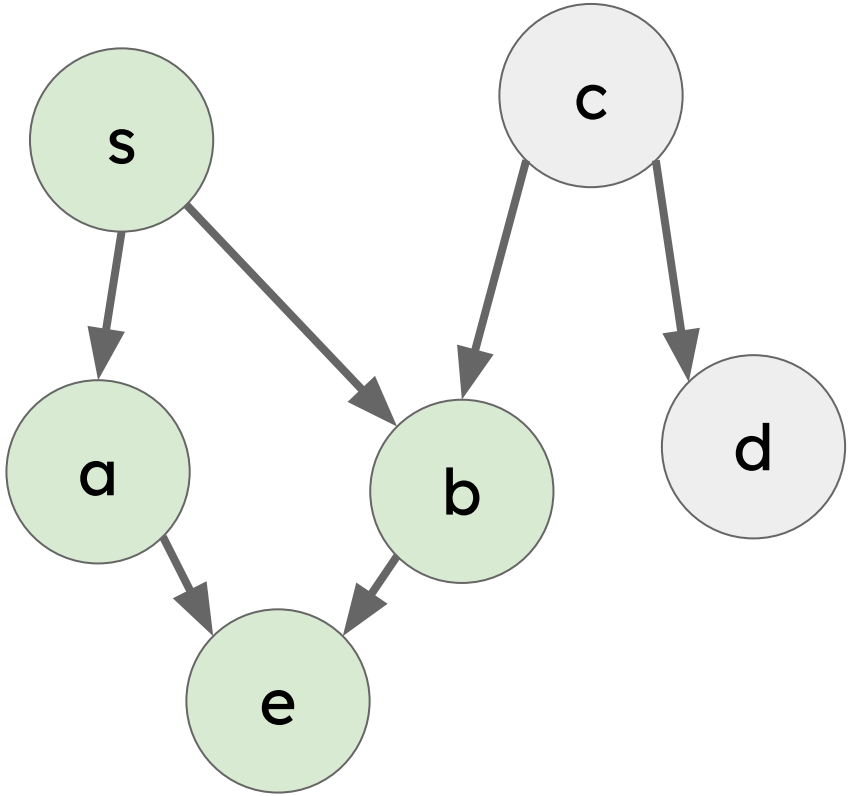
Remove the next item, which happens to be e again

BFS: [ ]



e is already marked as visited. Do nothing!

BFS: [ ]



Queue empty, return.

```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all false>
    queue = <a FIFO queue that holds only s>
    queue is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

What's the runtime?

Pop Quiz

BFS:

```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all f
    queue = <a FIFO queue that holds only s>
    queue is not empty:
        get some new node c_node out of queue
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

Again, we only visit each node at most once.

BFS:

```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all f
    queue = <a FIFO queue that holds only s>
    queue is not empty:
        get some new node c_node out of queue
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

Each time we do visit a node, we iterate deg(c_node) times.

```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised to all false>
    queue = <a FIFO queue that holds only s>
    queue is not empty:
        get some new node c_node out of queue
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours
    return visited_list
```

So this takes O(n + m) time.

```
traversal_from_s(G, s):
    visited_list = <an array of size n, initialised
    queue = <a FIFO queue that holds only s>
    queue is not empty:
        get some new node c_node out of queue
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return visited_list
```

Try to think about what happens if we used an edge list or adjacency matrix

Something to Try

Adapting Traversal Algorithms

How to modify the stuff you just saw

Adapting Graph Traversal Algorithms:

There are many graph problems that have the same core idea, but are distinct enough that you can't say "just DFS/BFS" and end there.

Essentially, you'd want the traversal, but you need to still insert a few lines here and there to actually properly solve the problem.

Example Problems:

- Cycle detection in undirected graphs
- Seeing if node t reachable from node s
- Counting the number of connected components in an undirected graph
- Cycle detection in directed graphs
- Shortest pathfinding in undirected/directed graphs

Even Further Beyond Graphs:

Even better is the fact that seemingly non-graph problems can often be turned into graph problems. For example:

- Minimum swaps needed to sort an array
- MS Paint Fill Feature
- Class size allocations
- Scheduling Problems
- Dependency tracking and resolution

Adapting Graph Traversal Algorithms:

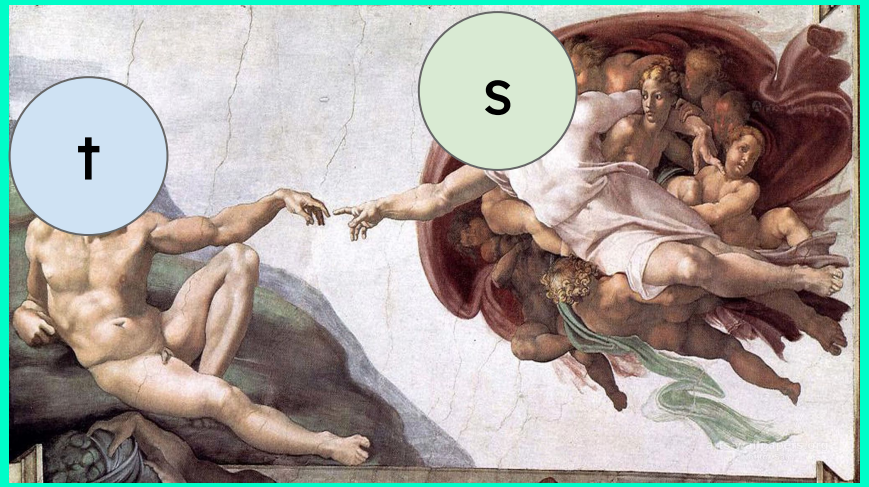
There are many graph problems that have the same core idea, but are distinct enough that you can't say "just DFS/BFS" and end there.

Essentially, you'd want the traversal, but you need to still insert a few lines here and there to actually properly solve the problem.

So instead of teaching you the solution to all those problems, we'll teach you how to adapt traversals to solve problems in general.

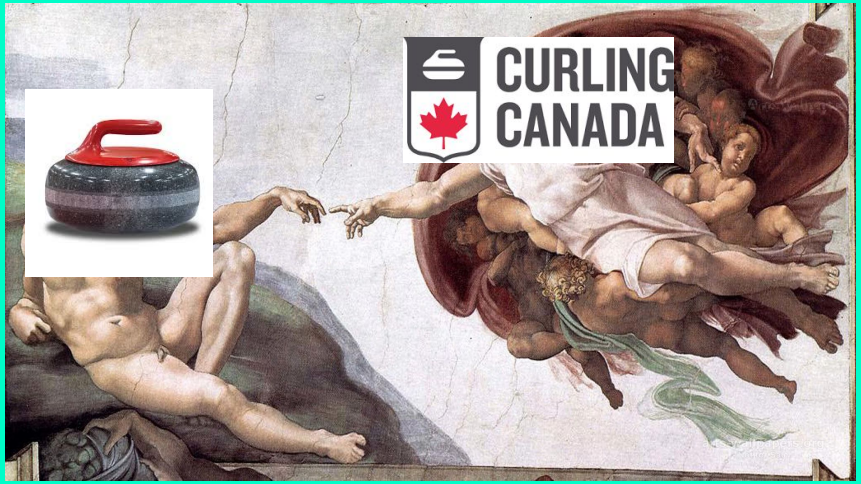
s-t reachability

Warmup 1 to adapting traversals



s-t reachability

Warmup 1 to adapting traversals



s-t reachability

Input: An directed graph G = (V, E), node id s, node id t.

Output: True if there exists a path from node s to node t.

s-t reachability

Idea: Start a traversal from node s, and see if node t is ever visited.

```
traversal_from_s(G, s, t):
    visited_list = <an array of size n, ir
    queue = <a FIFO queue that holds
    queue is not empty:
        get some new node c_node ou
        If c_node == t, return true
        If c_node has been visited before, skip this iteration
        Mark c_node as visited
        Add c_node's neighbours into the data structure
    return false
```

Key changes here:

1. Added termination condition
2. Changing what we return

Cycle detection in undirected graphs

Warmup 2 to adapting traversals

Undirected Cycle Detection

Input: An undirected graph G = (V, E)

Output: True if a cycle exists in the graph. False if it doesn't.

Undirected Cycle Detection

Idea 1: Do we even need to traverse?

Output true if m >= n, otherwise output false.

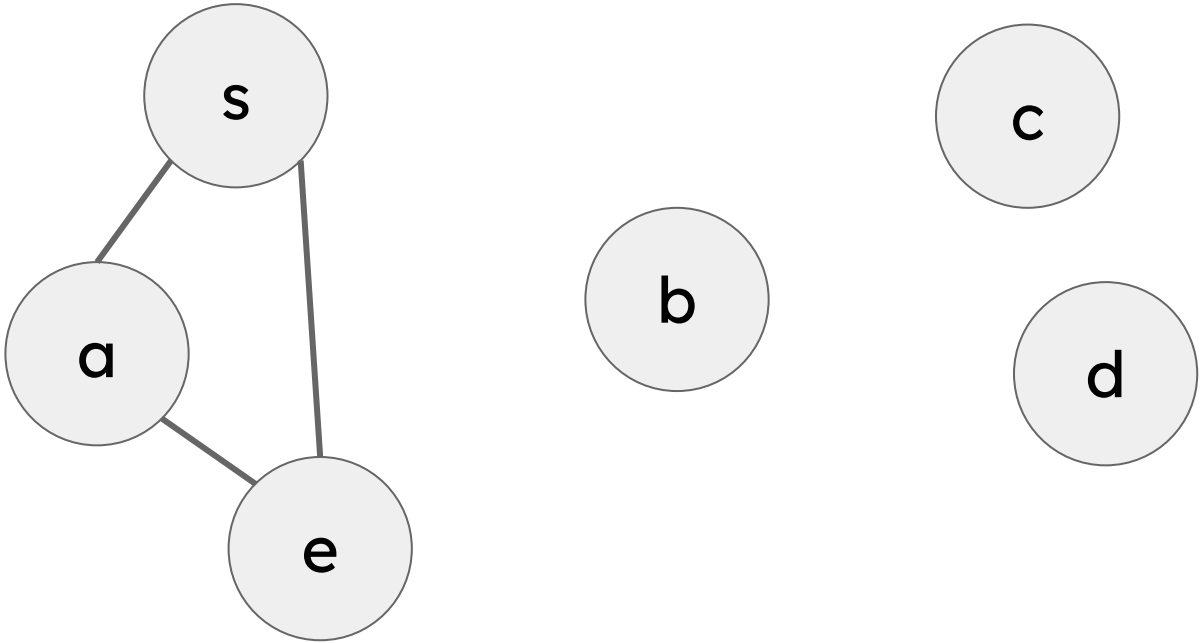
Undirected Cycle Detection

Idea 1: Do we even need to traverse?

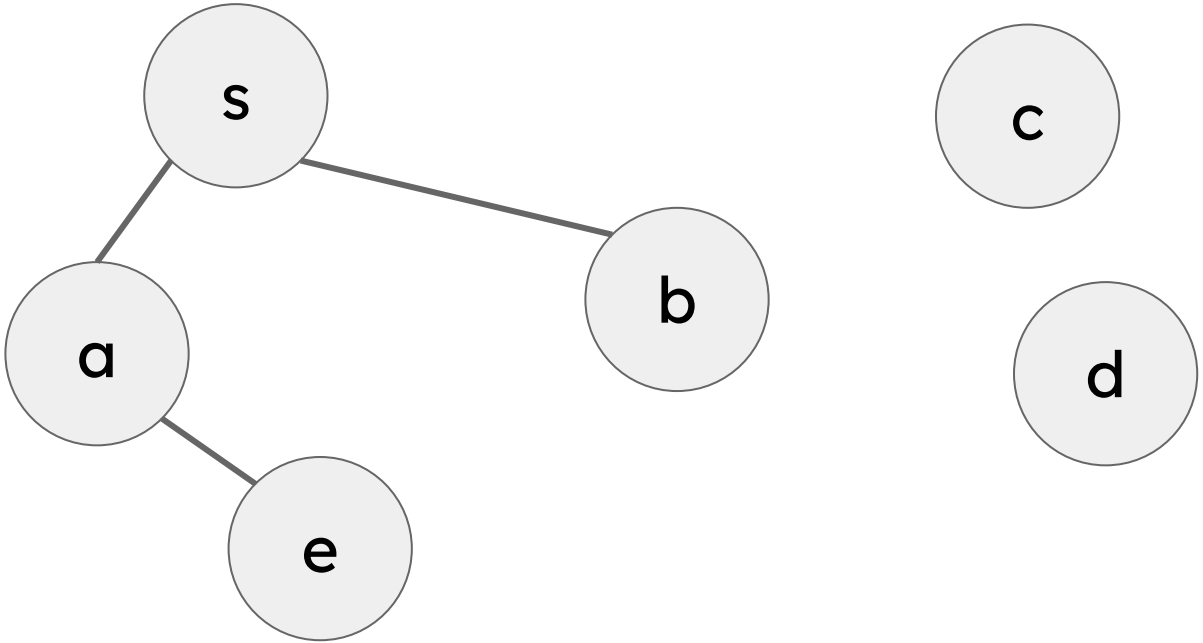
Output true if m >= n, otherwise output false.

Does this algo work?

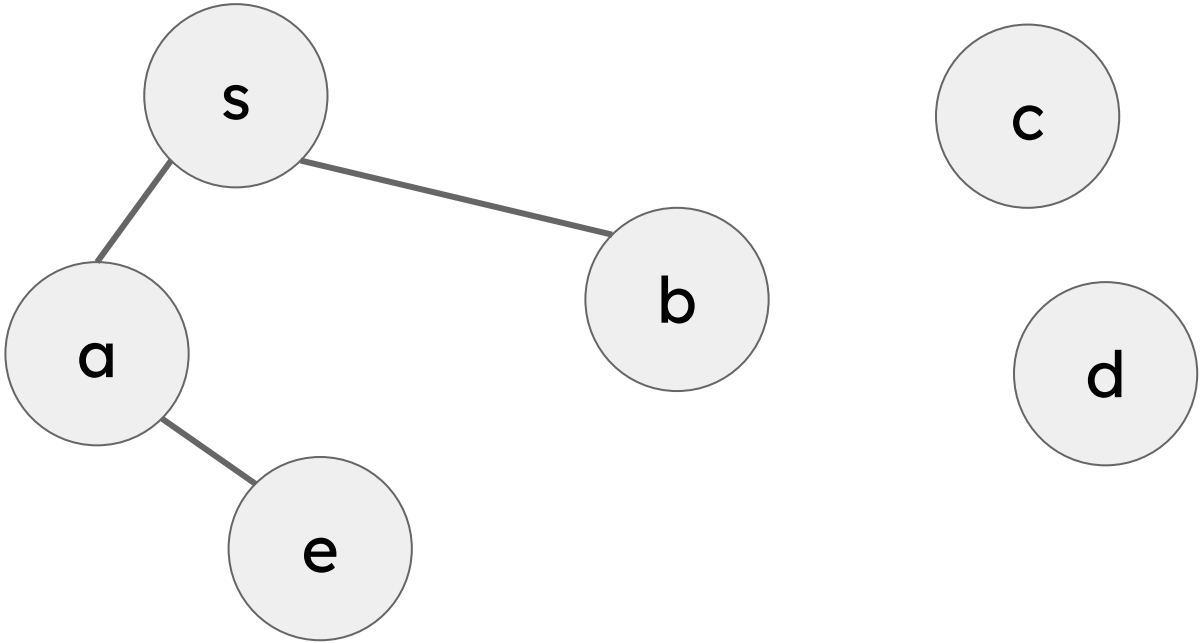
Pop Quiz



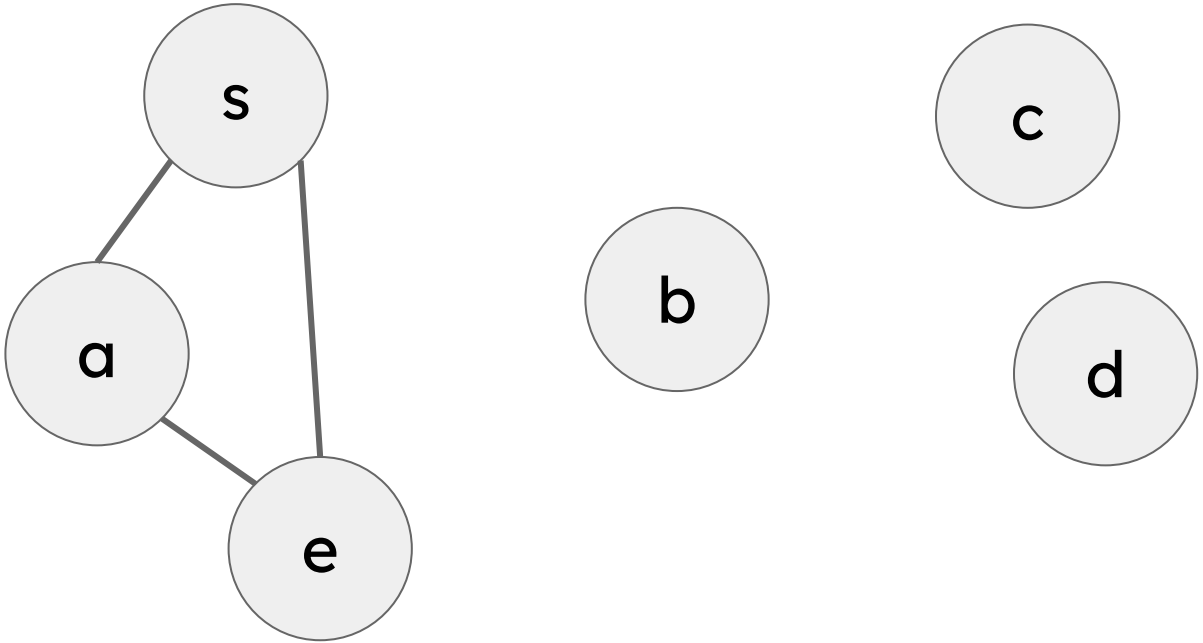
We have m = 3, but n = 6, but here there is indeed a cycle.



We have m = 3, but n = 6, but here there is no cycle.



If it is acyclic, then we know that m <= n - 1, but this is only a one-way implication.



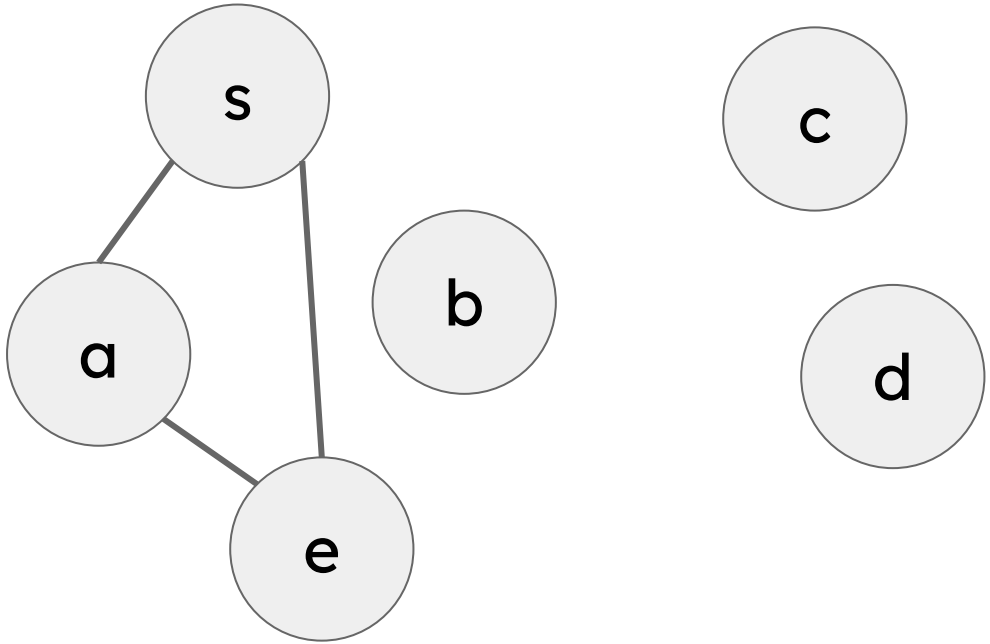
5. (a) Let $G = (V, E)$ be a simple, undirected graph. Prove that if G is acyclic, then $|E| \leq |V| - 1$.

   (b) Is the converse true?

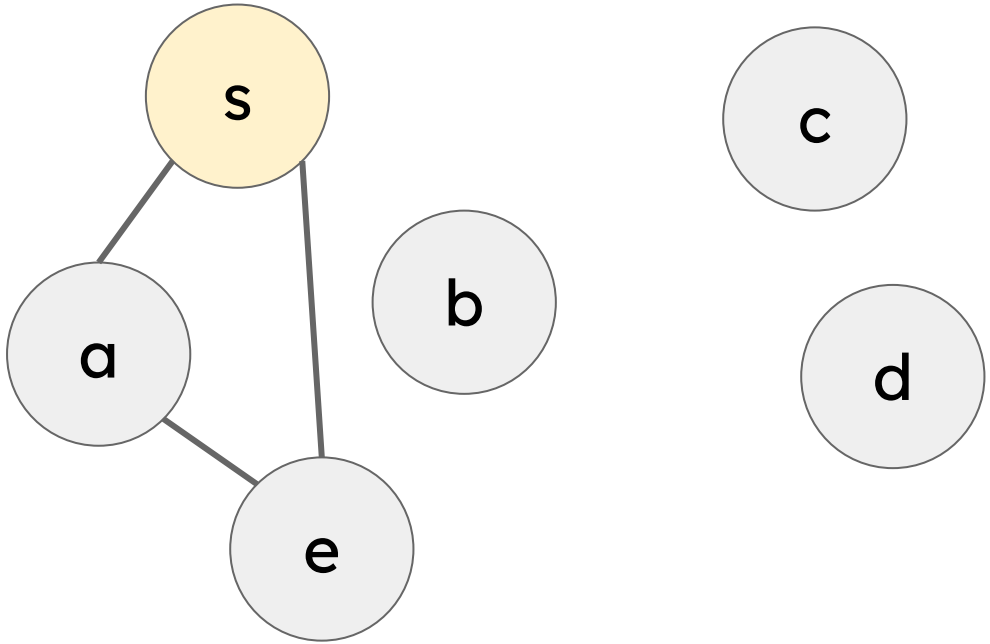
In fact, you should know this from CS1231S tutorial. 😈

Undirected Cycle Detection

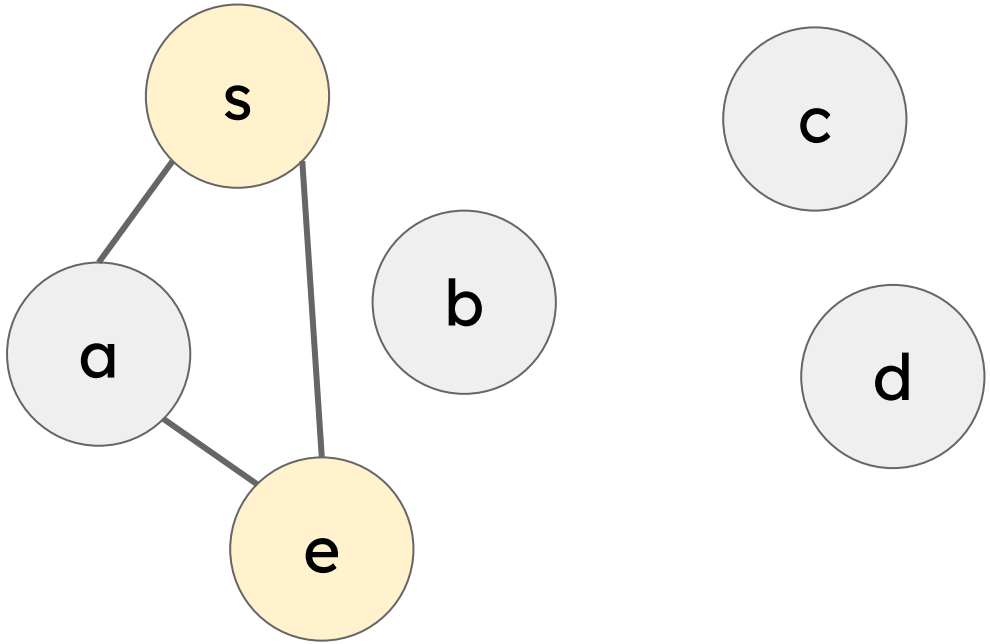
Idea 2: Let’s just traverse from a node, and if we revisit itself again, then we output true, otherwise, output false.



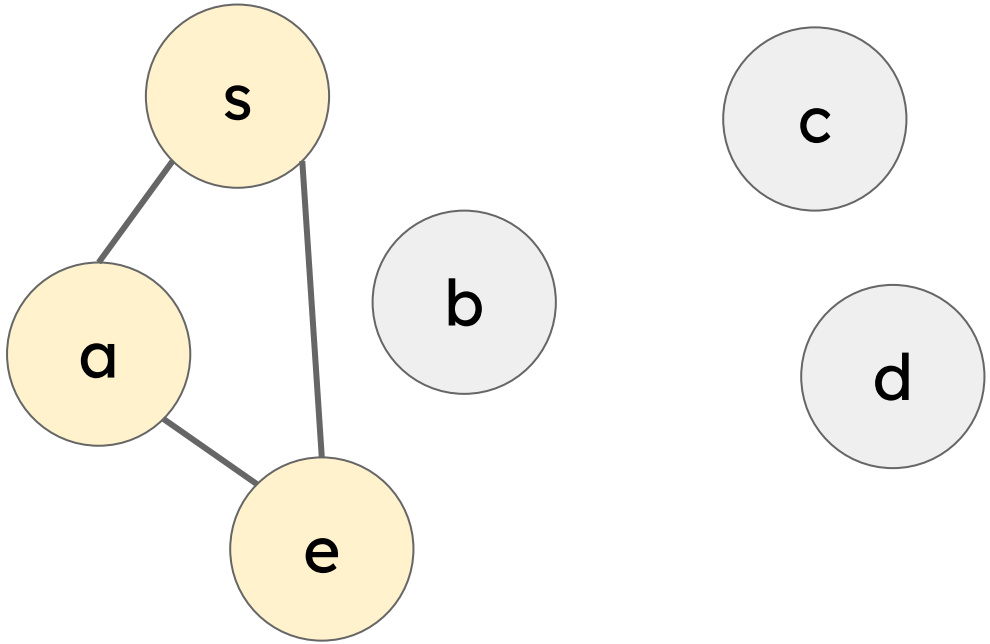
For example, if we started traversing from node s, (either via BFS or DFS)...



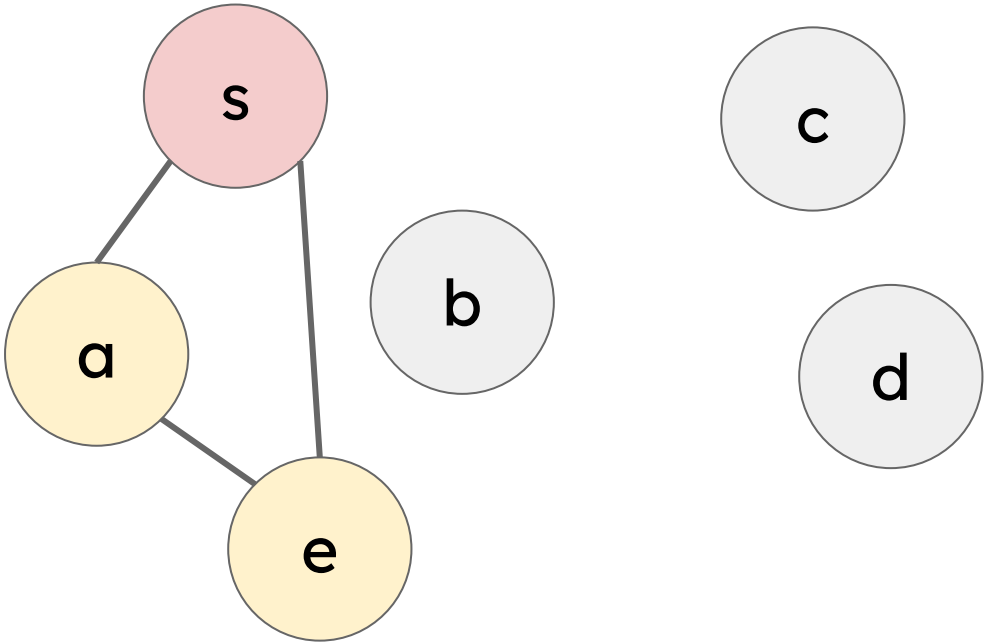
For example, if we started traversing from node s, (either via BFS or DFS)...



For example, if we started traversing from node s, (either via BFS or DFS)...



For example, if we started traversing from node s, (either via BFS or DFS)...



For example, if we started traversing from node s,
(either via BFS or DFS),
we would eventually revisit node s. Cycle detected!

Undirected Cycle Detection

Idea 2: Let's just traverse from a node, and if we revisit itself again, then we output true, otherwise, output false.

Question: Does this work?

Pop Quiz

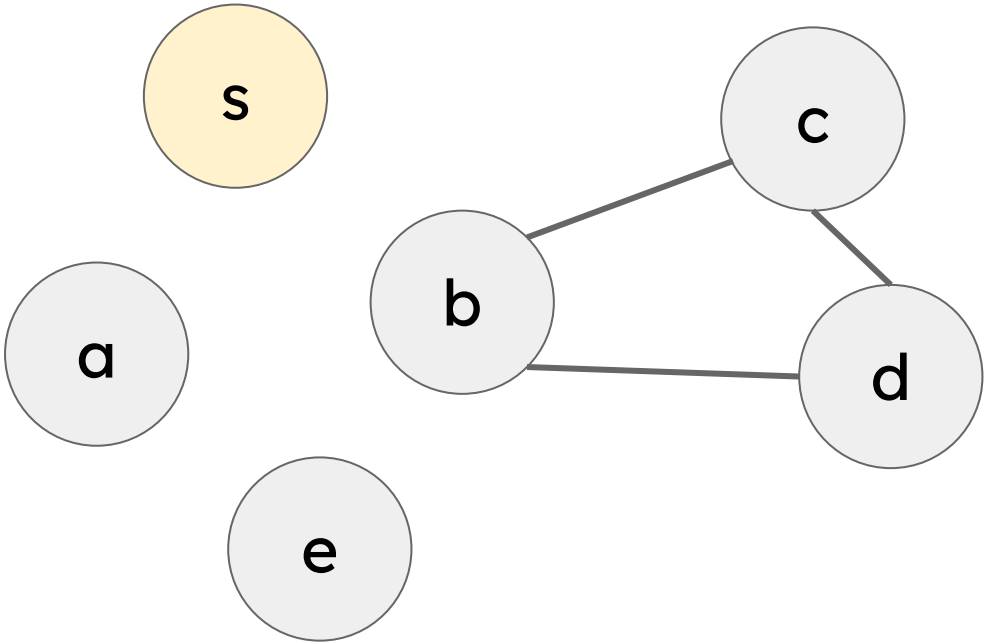
Undirected Cycle Detection

Idea 2: Let's just traverse from a node, and if we revisit itself again, then we output true, otherwise, output false.

There are two issues:

1. ???

2. ???



If we started traversal from node s here, we won’t see the cycle.

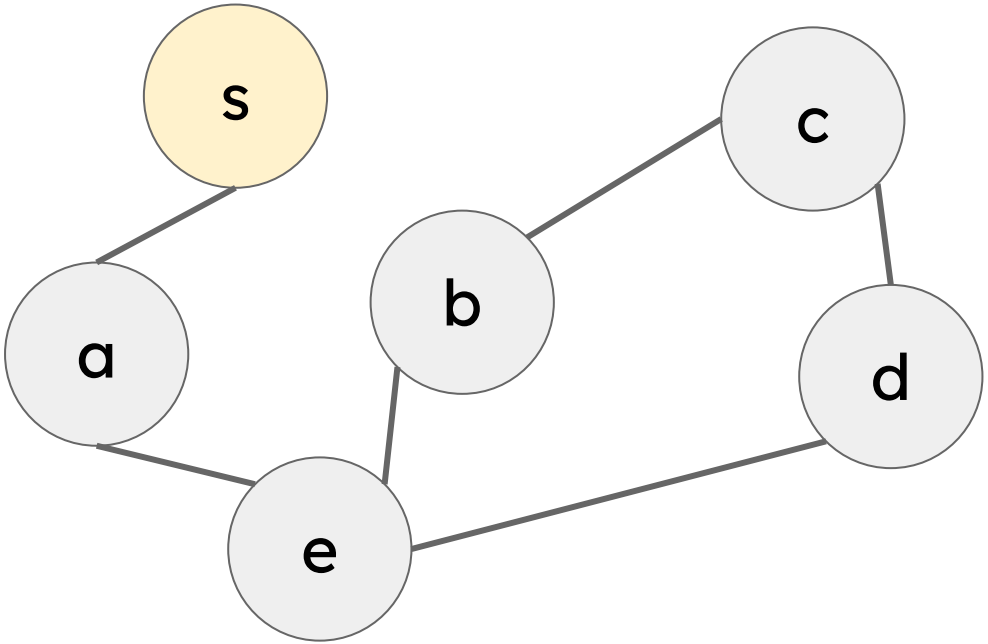
Undirected Cycle Detection

Idea 2: Let's just traverse from a node, and if we revisit itself again, then we output true, otherwise, output false.

There are two issues:

1. We need to start traversing from every node, not just start traversing a fixed node.

2. ???



If we started traversal from node s here, we will revisit some node in the traversal, but not node s itself. We should **still** report that a cycle exists.

Undirected Cycle Detection

Idea 2: Let's just traverse from a node, and if we revisit itself again, then we output true, otherwise, output false.

There are two issues:

1. We need to start traversing from every node, not just start traversing a fixed node.

2. Revisiting **any node** (not just s), is enough for us to report a cycle.

```
traversal_from_s(G, s):
    visited_list = [ all false ]
    queue = [ a FIFO queue that only holds s
    data structure is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, return true
        Mark c_node as visited
        Add c_node's neighbours into the data structure
```

Key modification here!

Then run:

traversal_from_s(G, v) for all nodes v in V.

```
traversal_from_s(G, s):
    visited_list = [ all false ]
    queue = [ a FIFO queue that only holds s
    data structure is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, return true
        Mark c_node as visited
        Add c_node's neighbours into the data structure
```

What's the runtime?

Then run:

traversal_from_s(G, v) for all nodes v in V.

Pop Quiz

```
traversal_from_s(G, s):
    visited_list = [ all false ]
    queue = [ a FIFO queue that only holds s ]
    data structure is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, return true
        Mark c_node as visited
        Add c_node's neighbours into the data structure
```

Observations:

1. We visit each node at most once.
2. We traverse each edge at most once.

traversal_from_s(G, s):

visited_list = [all false]

Is that true?

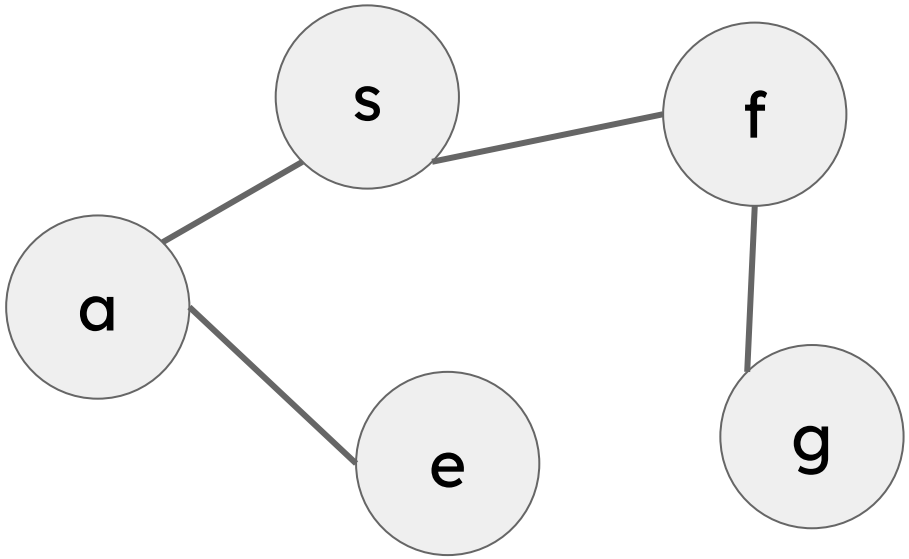
If c_node has been visited before, return true

Mark c_node as visited

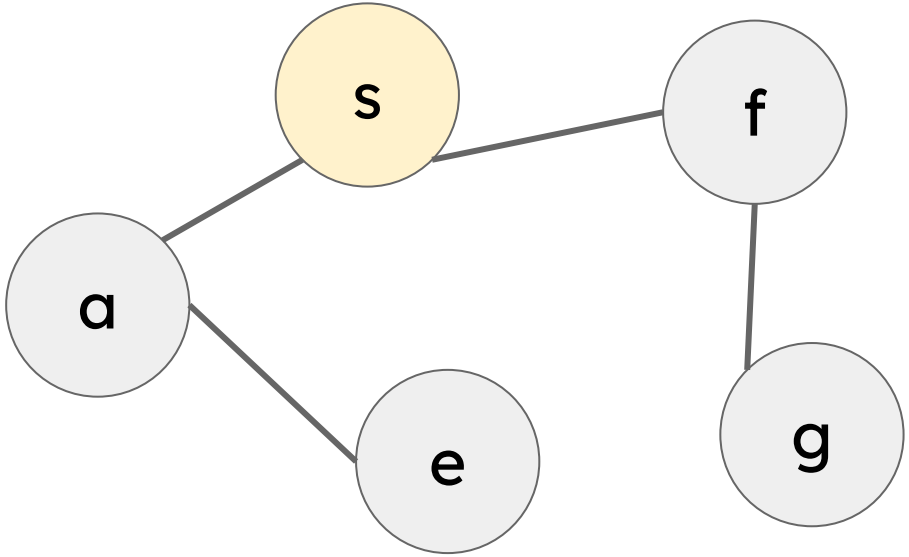
Add c_node's neighbours into the data structure

Observations:

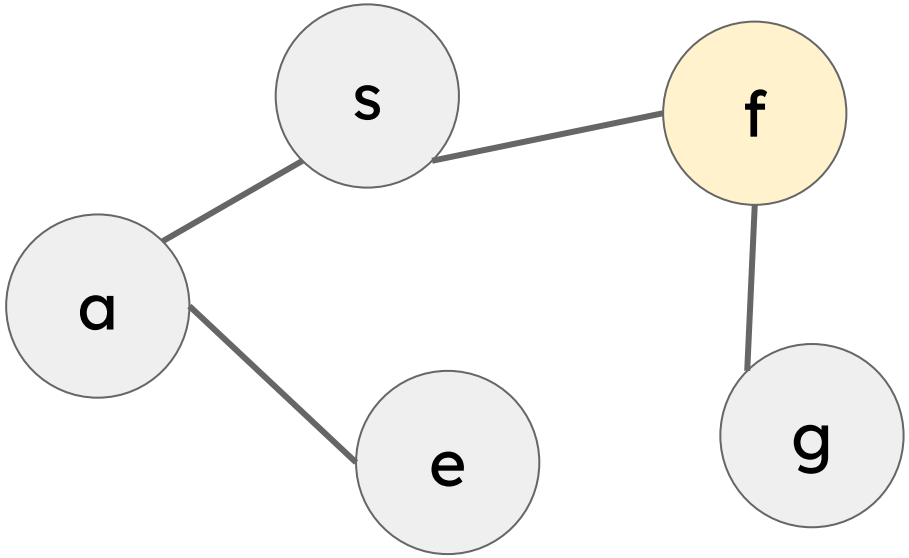
1. We visit each node at most once.
2. We traverse each edge at most once.



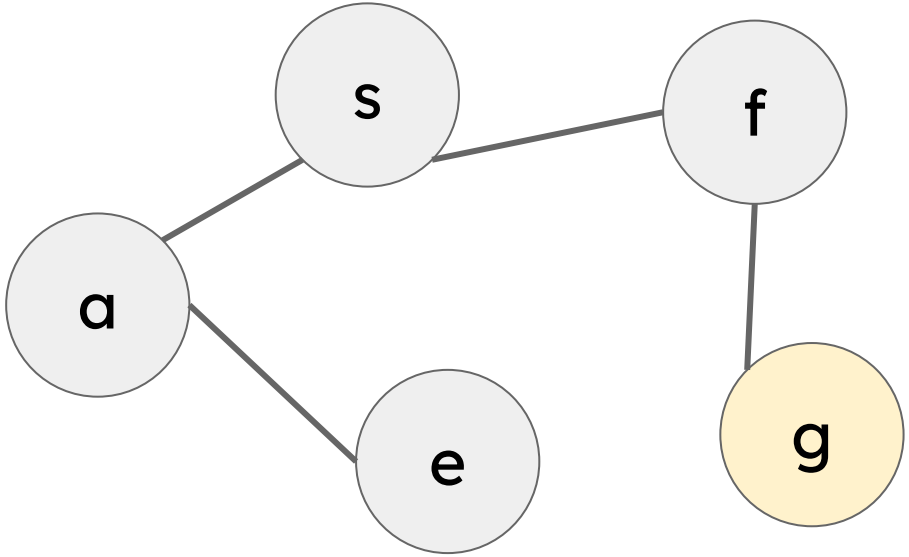
Every time we start a traversal, we're forgetting which nodes we already visited. And doing a fresh traversal all over again.



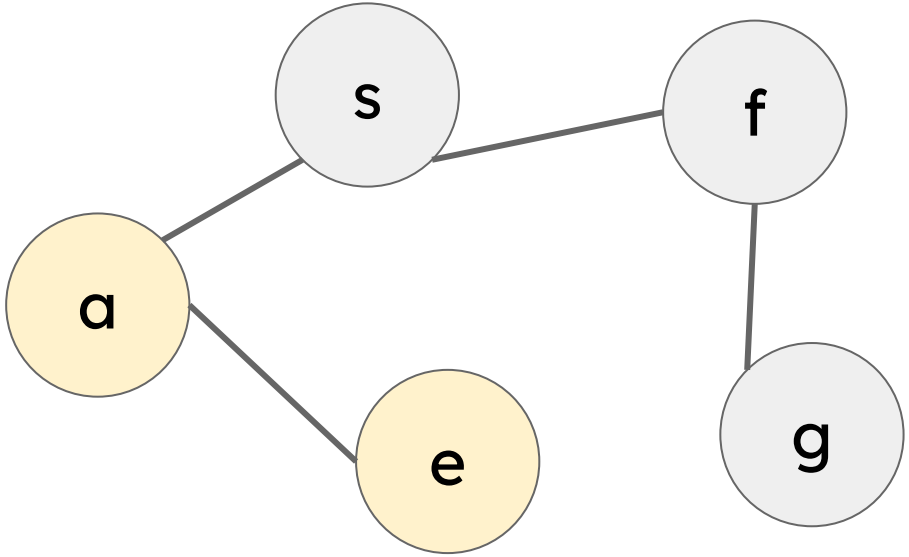
This traversal is going to cost O(n) time…



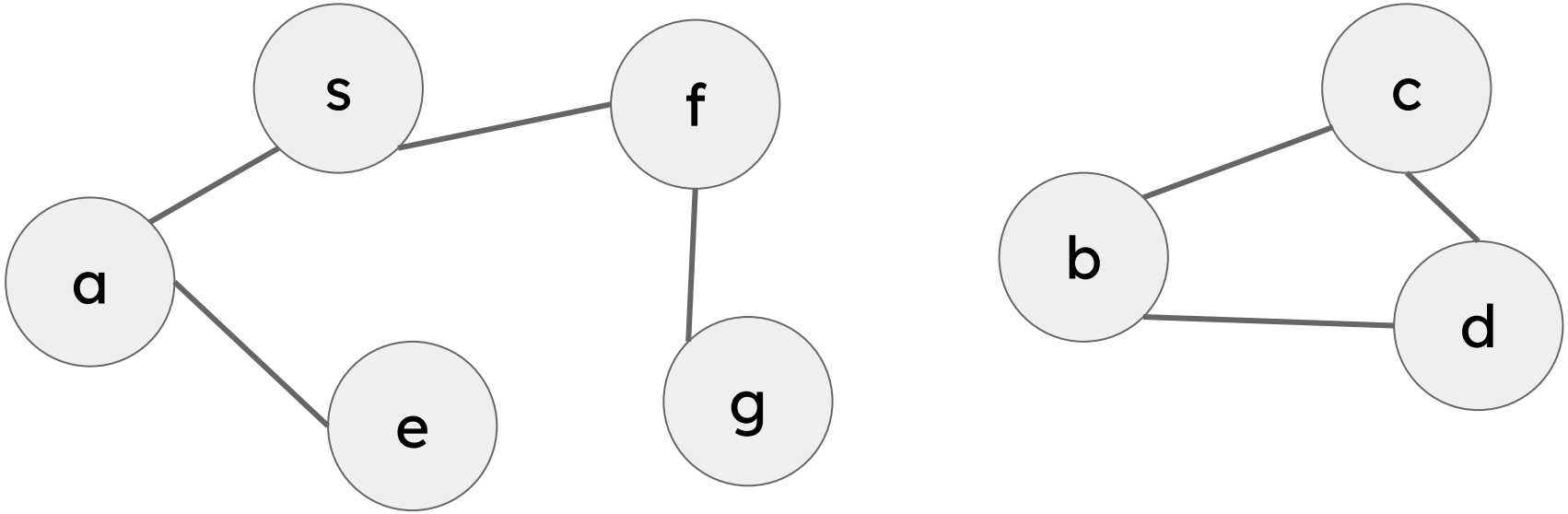
Same for this traversal…



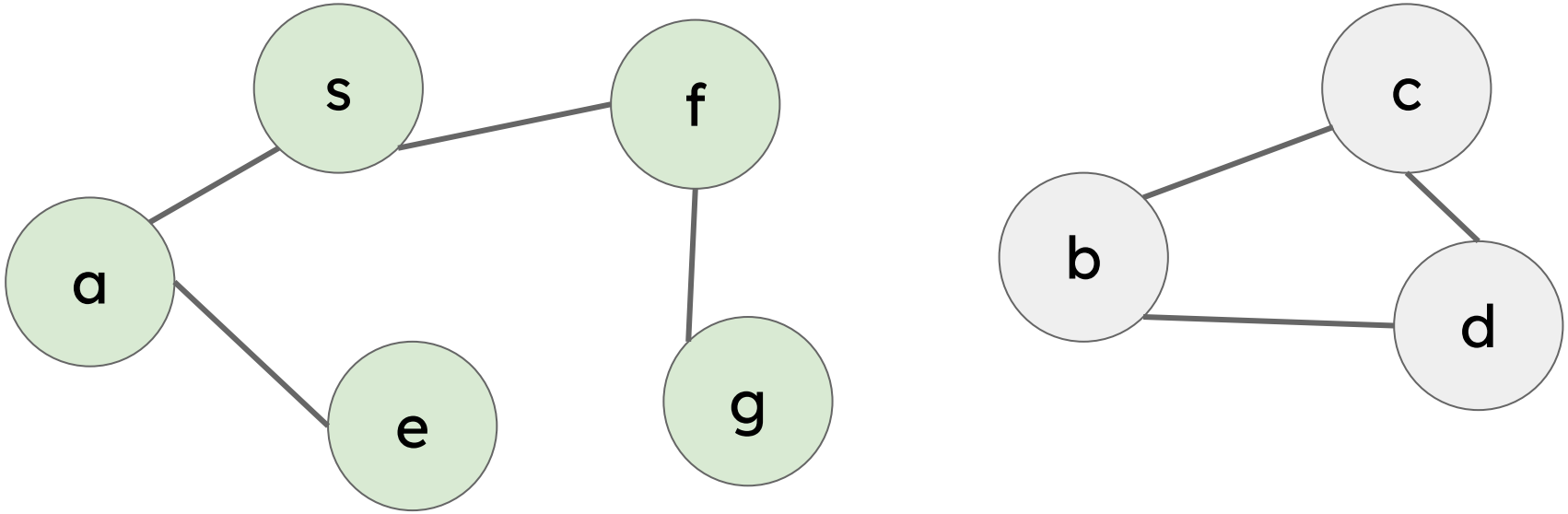
Same as well for this one…



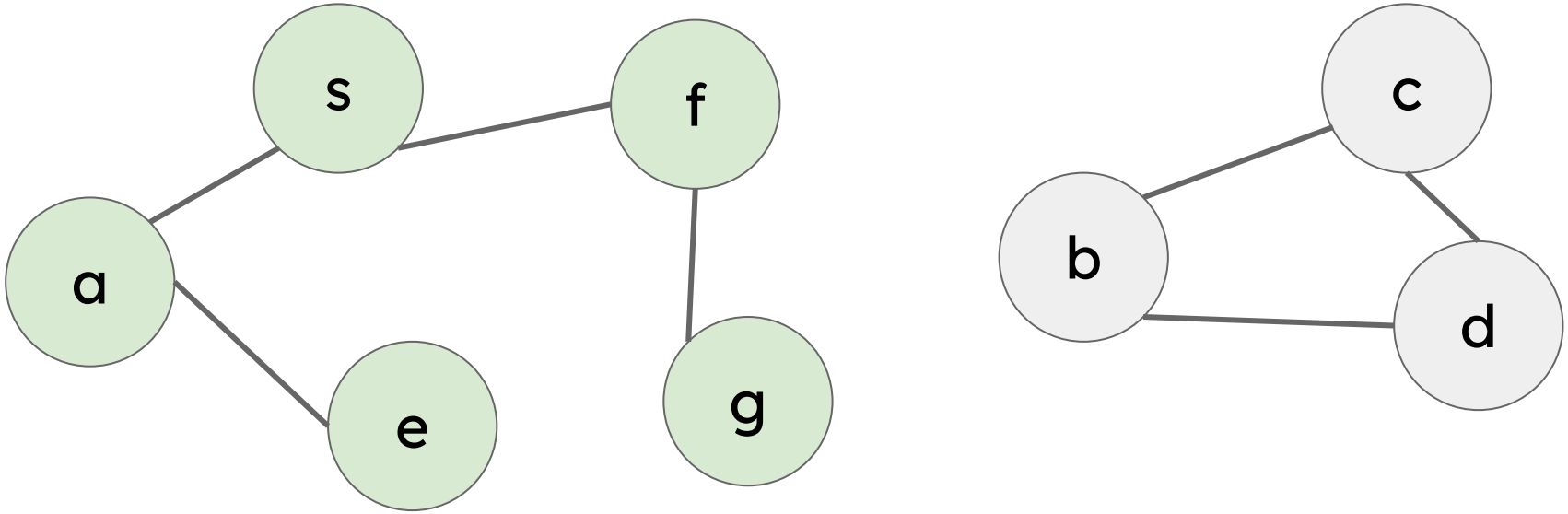
And so on for the rest.



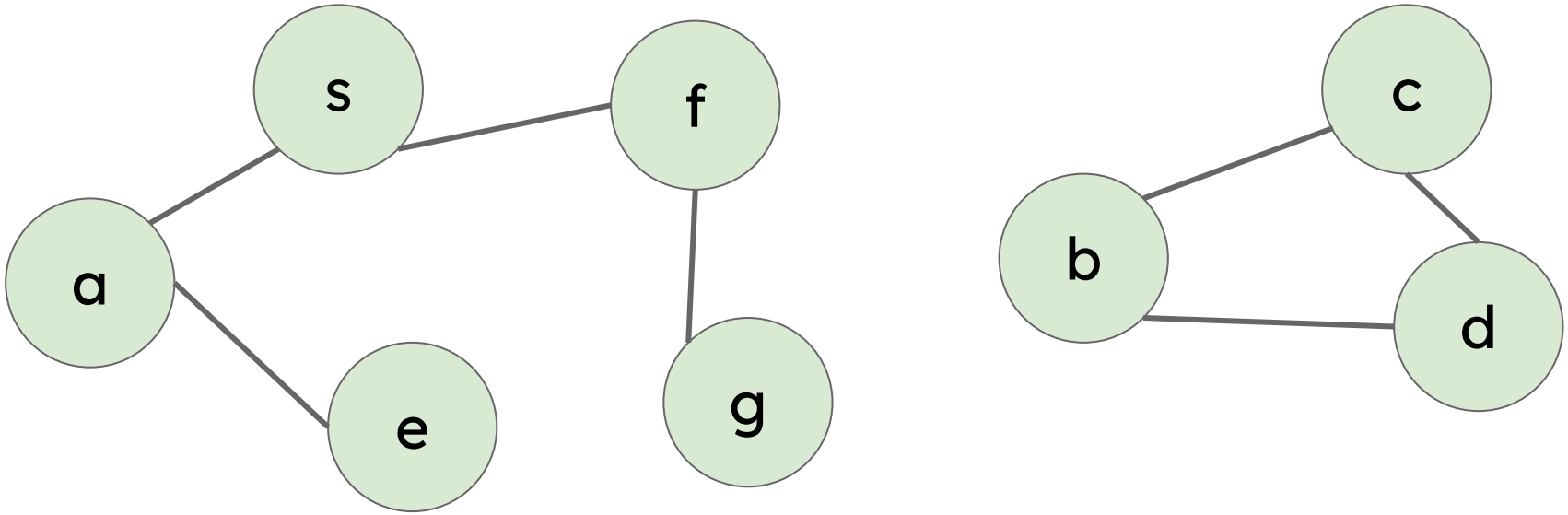
To fix this, we need to remember the nodes that we have visited before. But also try our best to start traversals from unvisited nodes.



For example, one BFS/DFS from node s should mark out all nodes in its connected component.



So if we tried to start a new BFS/DFS from any of those nodes, we'd return straight away.



Then the moment we try a traversal again from e.g. node b, it should again mark all nodes in its connected component as visited.

```
traversal_from_s(G, s, visited_list):
    queue = [ a FIFO queue that only holds
    data structure is not empty:
        get some new node c_node out of
        If c_node has been visited before,
        Mark c_node as visited
        Add c_node's neighbours into the d
```

Let's move the visited_list as a parameter of the function.

```
traversal_from_s(G, s, visited_list):
    queue = [ a FIFO queue that only holds s ]
    data structure is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, return true
        Mark c_node as visited
        Add c_node's neighbours into the data structure
```

Then create visited_list = [array of all false]. Then run traversal_from_s(G, v, visited_list) for all nodes v in V.

```
traversal_from_s(G, s, visited_list):
    queue = [ a FIFO queue that only holds
    data structure is not empty:
        get some new node c_node out of
        If c_node has been visited before,
        Mark c_node as visited
        Add c_node's neighbours into the d
```

This way, in between traversals, we remember which nodes were visited already, and which were not.

Then create visited_list = [array of all false]. Then run

traversal_from_s(G, v, visited_list) for all nodes v in V.

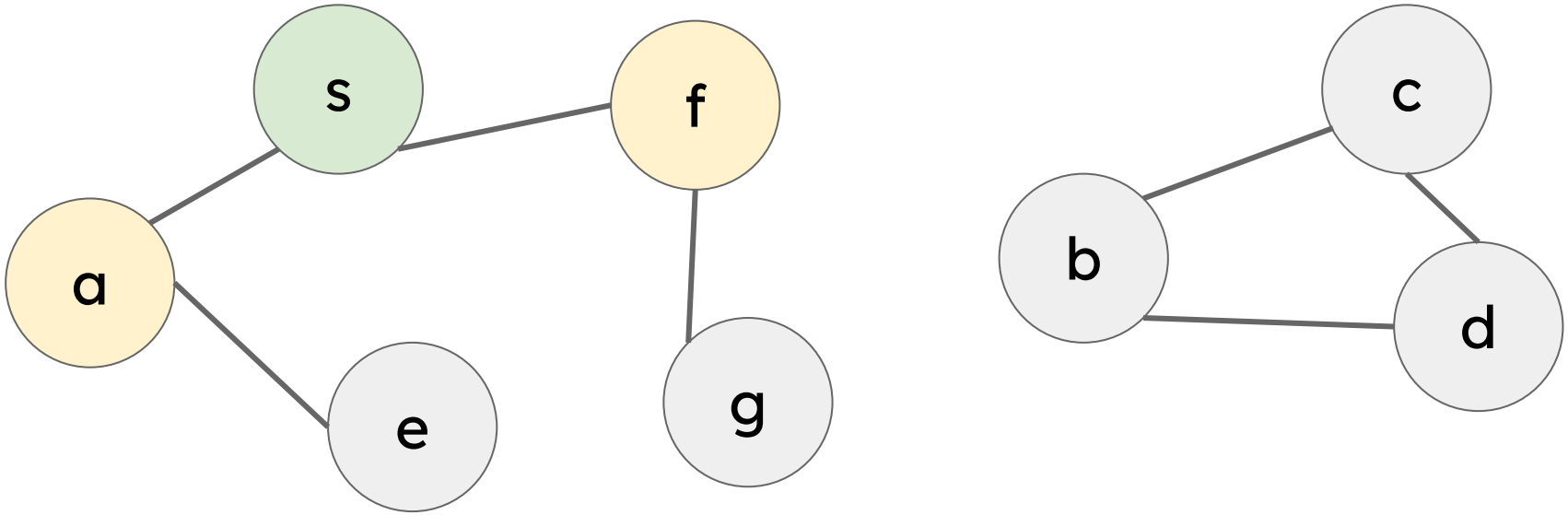
```
traversal_from_s(G, s, visited_list):
    queue = [ a FIFO queue that only holds
    data structure is not empty:
        get some new node c_node out of
        If c_node has been visited before,
        Mark c_node as visited
        Add c_node's neighbours into the d
```

If any call returns true, we know a cycle exists.

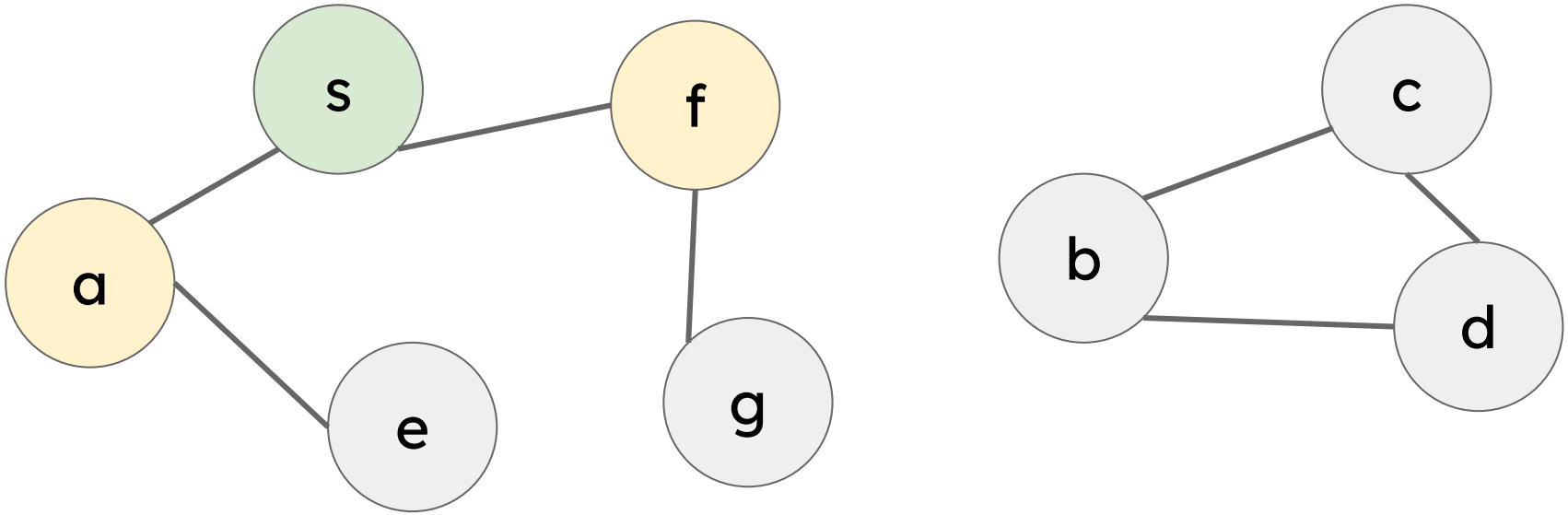
O(n + m) time.

Then create visited_list = [array of all false]. Then run

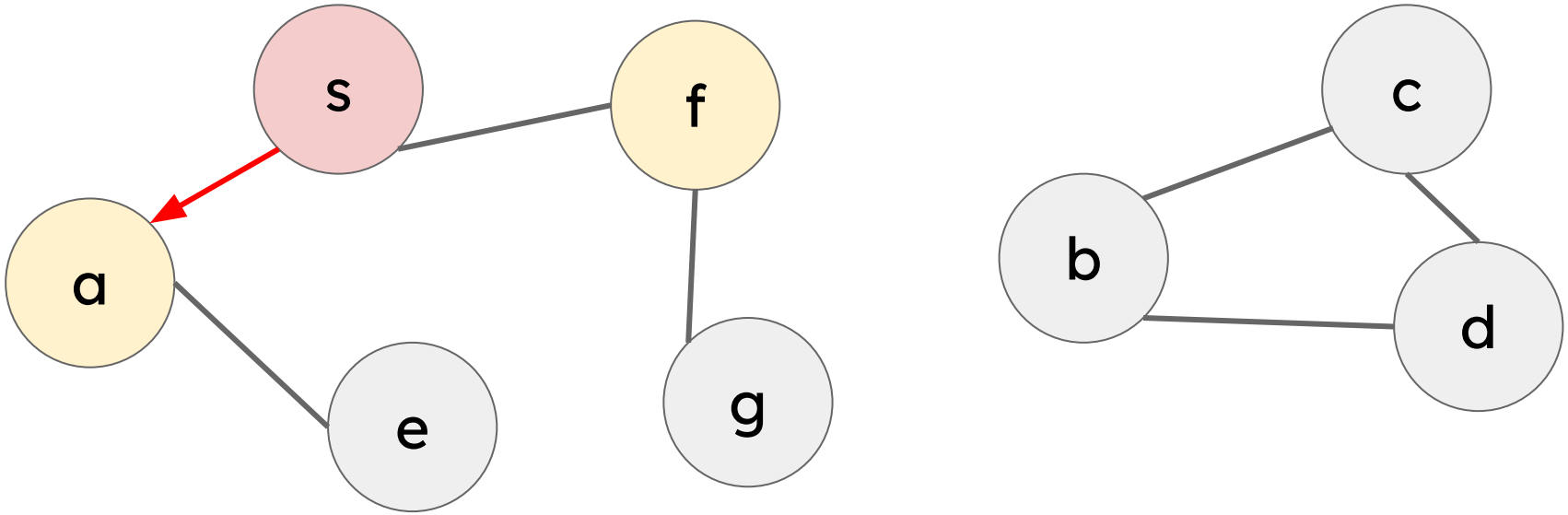
traversal_from_s(G, v, visited_list) for all nodes v in V.



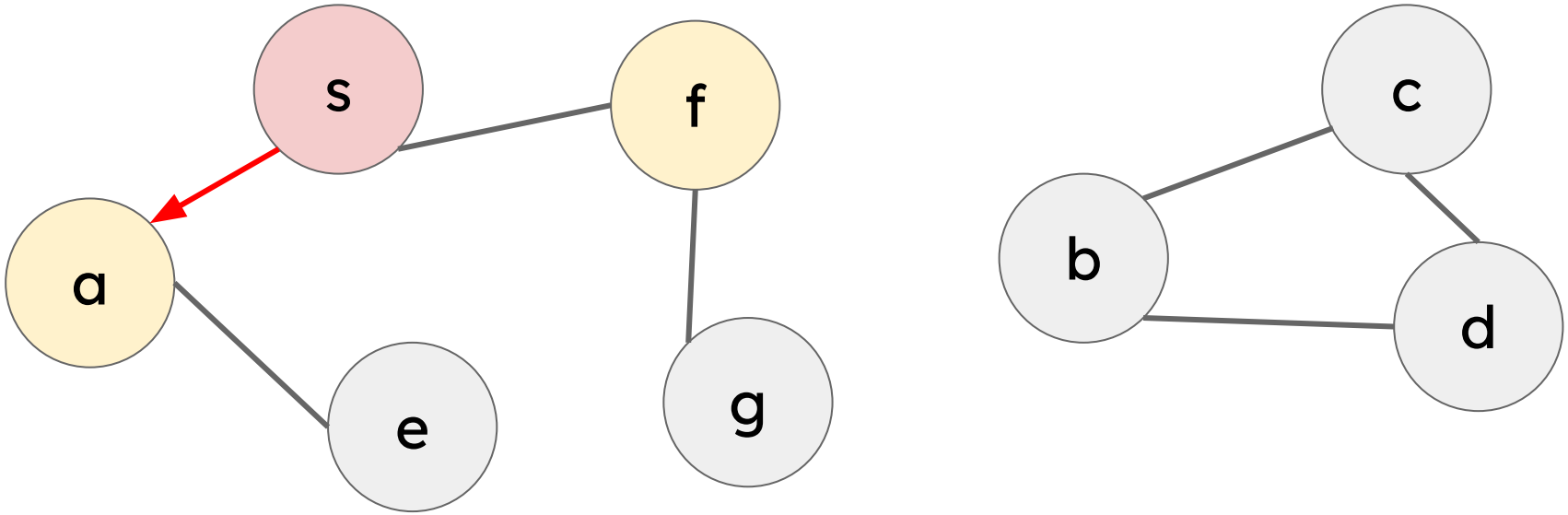
Except when we start the traversal from node s, we enqueue nodes a, f.



Then when we process node a, we look at nodes [s, e] again.



Even though s was visited, we don’t want to mark it as having a cycle. We just came from s after all.



Idea: remember for every node, how we visited it. Then if we see that neighbour, we ignore it.

```
traversal_from_s(G, s, visited_list):
    visited_by = [an array of size n]
    queue = [ a FIFO queue that only holds s ]
    data structure is not empty:
        get some new node c_node out of data structure
        For each neighbour n of c_node:
            If visited_by[c_node] = n continue
            visited_by[n] = c_node
            if visited return true
```

Handles the bidirectional edge case

```
traversal_from_s(G, s, visited_list):
    visited_by = [an array of size n]
    queue = [ a FIFO queue that only holds s ]
    data structure is not empty:
        get some new node c_node out of data structure
        For each neighbour n of c_node:
            If visited_by[c_node] = n continue
            visited_by[n] = c_node
            if visited return true
```

Marks our neighbour as visited by us

```
traversal_from_s(G, s, visited_list):
    visited_by = [an array of size n]
    queue = [ a FIFO queue that only holds s ]
    data structure is not empty:
        get some new node c_node out of data structure
        For each neighbour n of c_node:
            If visited_by[c_node] = n continue
            visited_by[n] = c_node
            if visited return true
            add n into the queue
```

Add in our unvisited neighbour

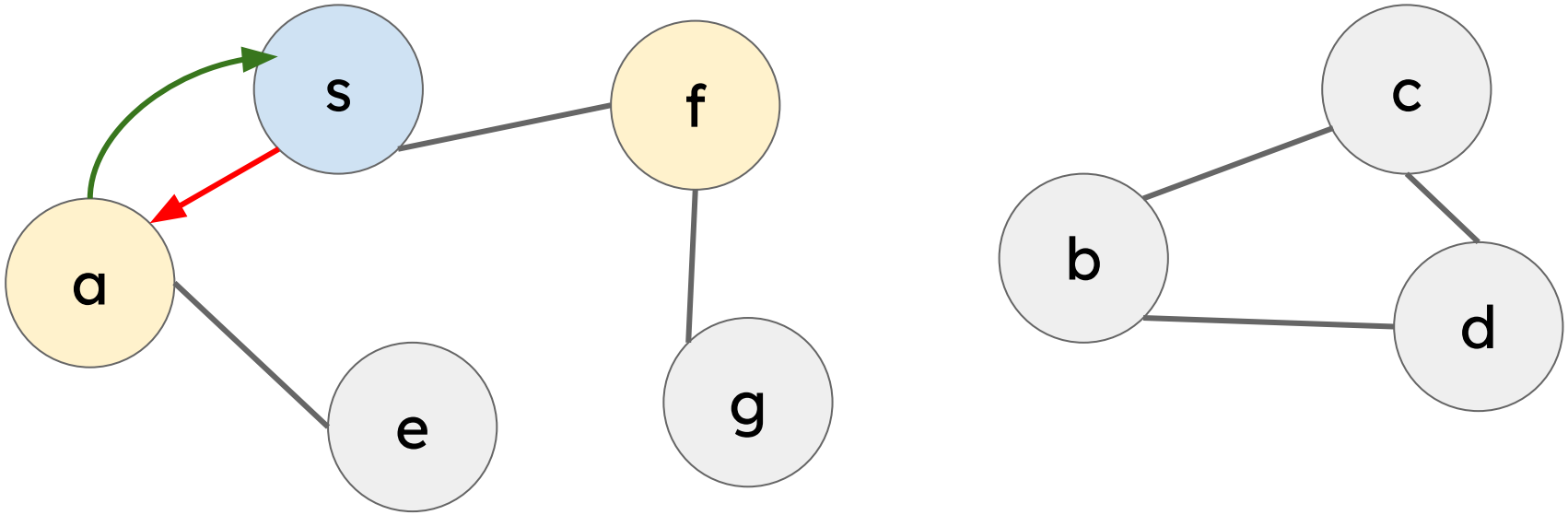
```
traversal_from_s(G, s, visited_list):
    visited_by = [an array of size n]
    queue = [ a FIFO queue that only holds s ]
    data structure is not empty:
        get some new node c_node out of data structure
        For each neighbour n of c_node:
            If visited_by[c_node] = n continue
            visited_by[n] = c_node
            if visited return true
            add n into the queue; mark n as visited
```

Mark it as visited as well

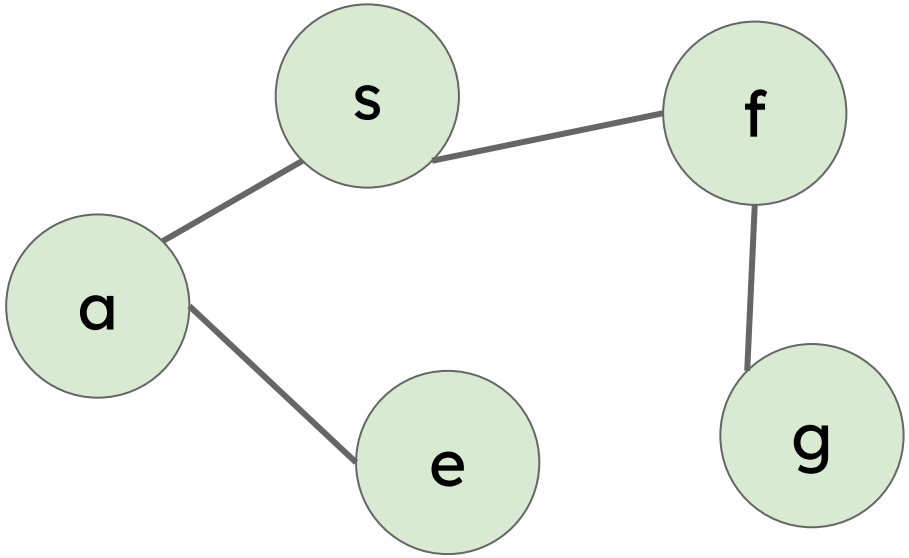
```
traversal_from_s(G, s, visited_list):
    queue = [ a FIFO queue that only holds s ]
    data structure is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, return true
        Mark c_node as visited
        Add c_node's neighbours into the data structure
```

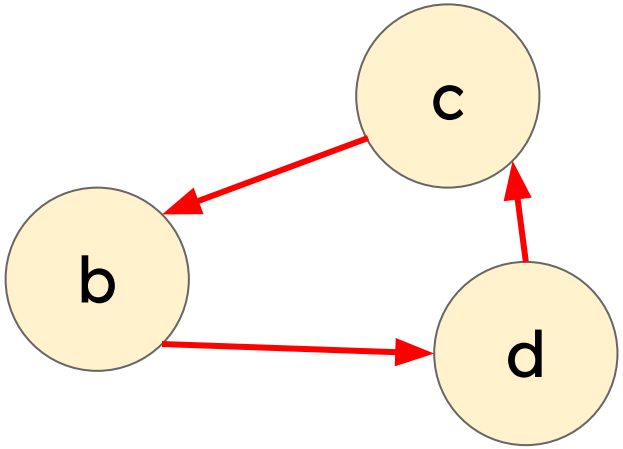
Then create visited_list = [array of all false]. Then run

traversal_from_s(G, v, visited_list) for all nodes v in V.



So this way, on node a, when we see node s again, we know it's the same edge we took (by using the visited_by) array, and we dont count it as part of a cycle.





On the other hand on a real cycle, visited_by[c] is not d. So we know it's a real cycle! Now we can return true.

Flood-fill / Graph Colouring

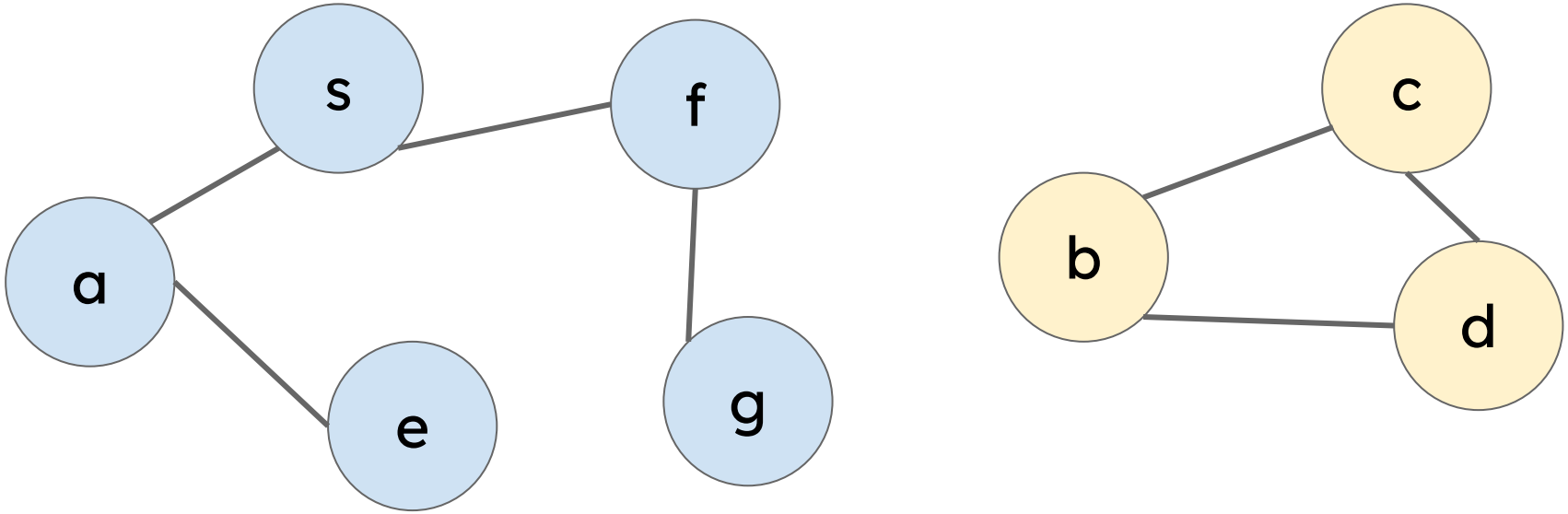
And how to count the number of connected components

Graph Colouring

Input: An undirected graph G = (V, E)

Output: Output an array `component_id` of n integers, such that:

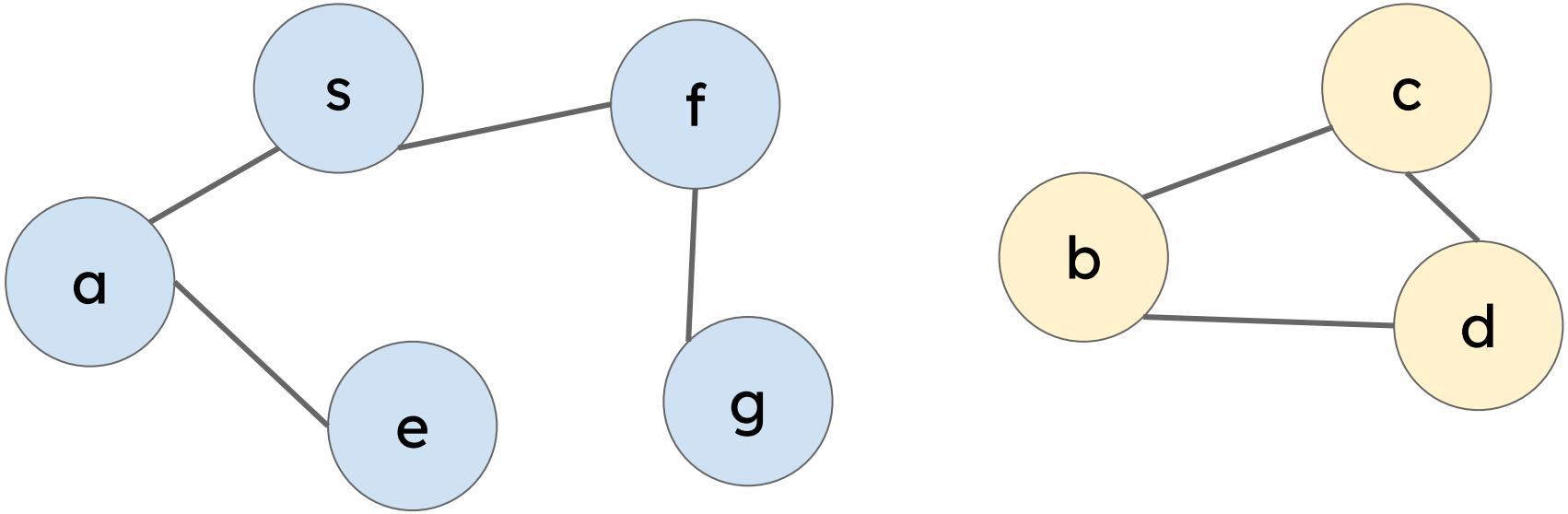
nodes u and v belong in the same connected component **if and only if** `component_id[u] = component_id[v]`



For example:

Nodes a, s, e, f, g should be given the same number.

Nodes b, c, d should be given the same number.



Furthermore, the numbers across connected components should be different.

Graph Colouring

Idea: Why not just re-use the same algorithm as before, but:

1. Before we start, we set a current_id = 1

2. Every time we need to do a fresh instance of a BFS/DFS, we set every node in that component to have current_id.

   Once we are done, we increment current_id by 1.

```
traversal_from_s(G, s, visited_list, component_id, current_id):
    queue = [ a FIFO queue that only holds s ]
    data structure is not empty:
        get some new node c_node out of data structure
        If c_node has been visited before, continue
        Mark c_node as visited, set component_id[c_node] = current_id
        Add c_node's neighbours into the data structure
```

Make sure the function takes in an array component_id.

We also need to know the current_id.

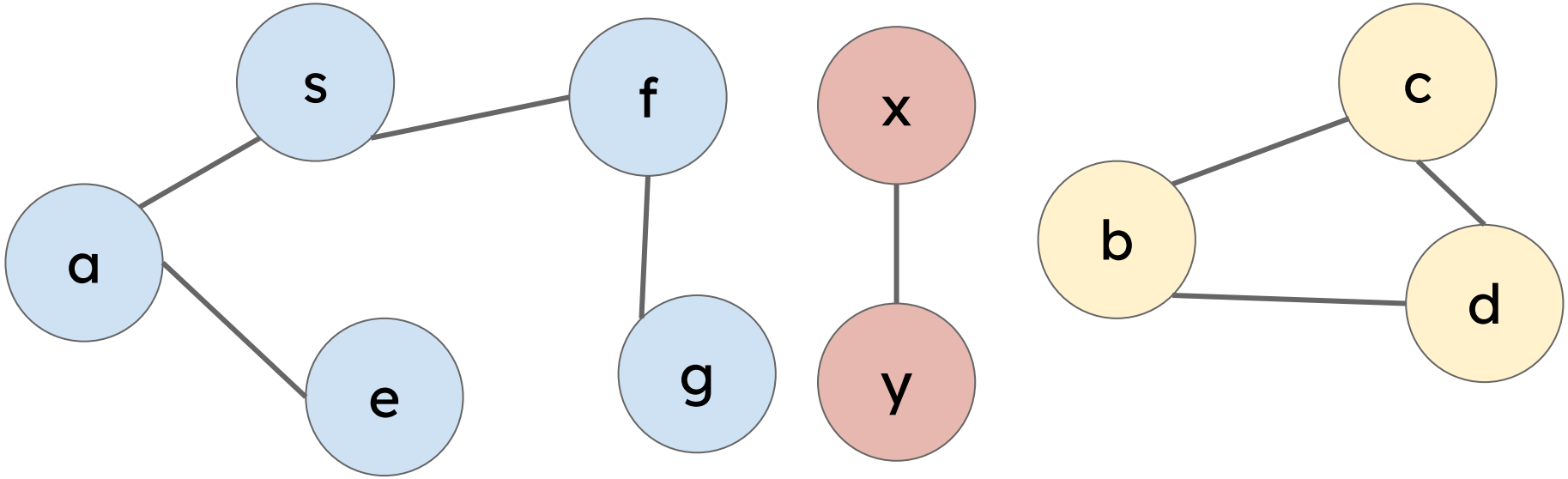
```
current_id = 1

component_id = <array of size n>

visited_list = <array of size n, initialised to all false>

for s in node list:

    if s is visited: continue

    traversal_from_s(G, s, visited_list, component_id, current_id)

    ++current_id
```

Then create the needed arrays, and repeated call our traversal algorithm.

Counting Components:

Input: An undirected graph G = (V, E)

Output: Count the number of connected components in a graph.



For example, for this graph, output 3.

Counting Components:

Idea: Why not just run flood-fill/graph colouring, and output the maximum possible id?

```
current_id = 1
component_id = <array of size n>
visited_list = <array of size n, initialised to all false>
for s in node list:
    if s is visited: continue
    traversal_from_s(G, s, visited_list, component_id, current_id)
    ++current_id
return current_id - 1
```

Just return the final current_id - 1!

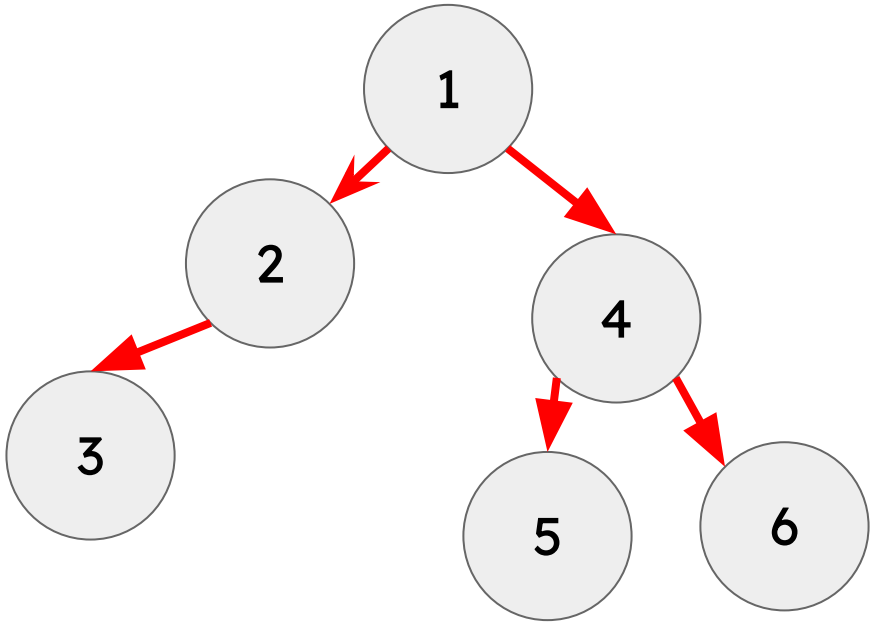
Moral of the Story

What you need to take away from this

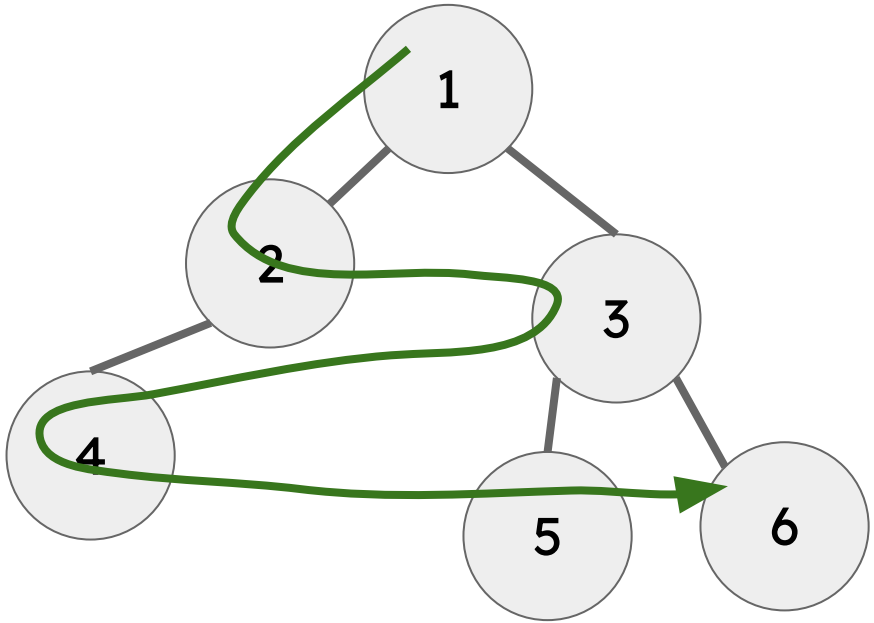
You should adapt, like Makora

BFS vs DFS

We've shown you two different traversal schemes so far.

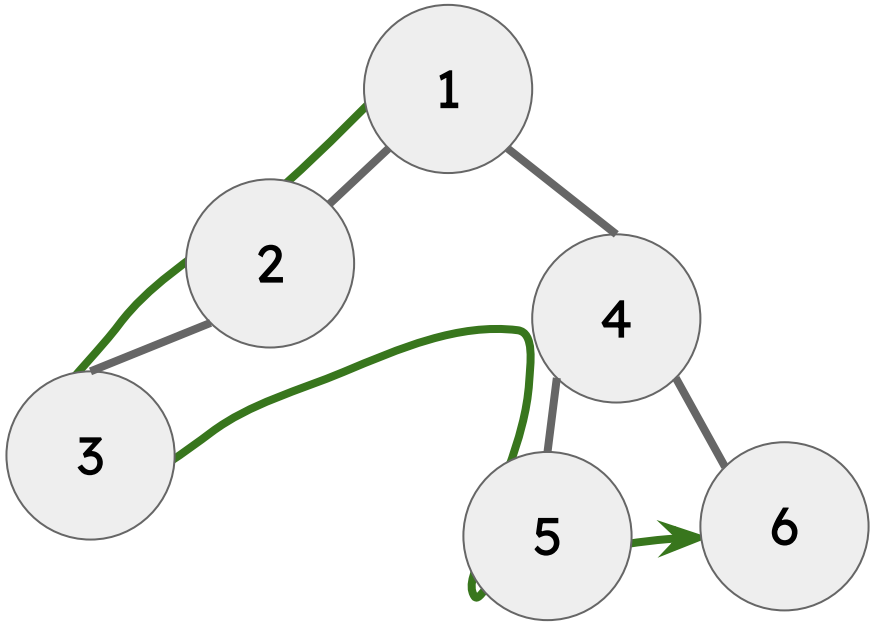


DFS:

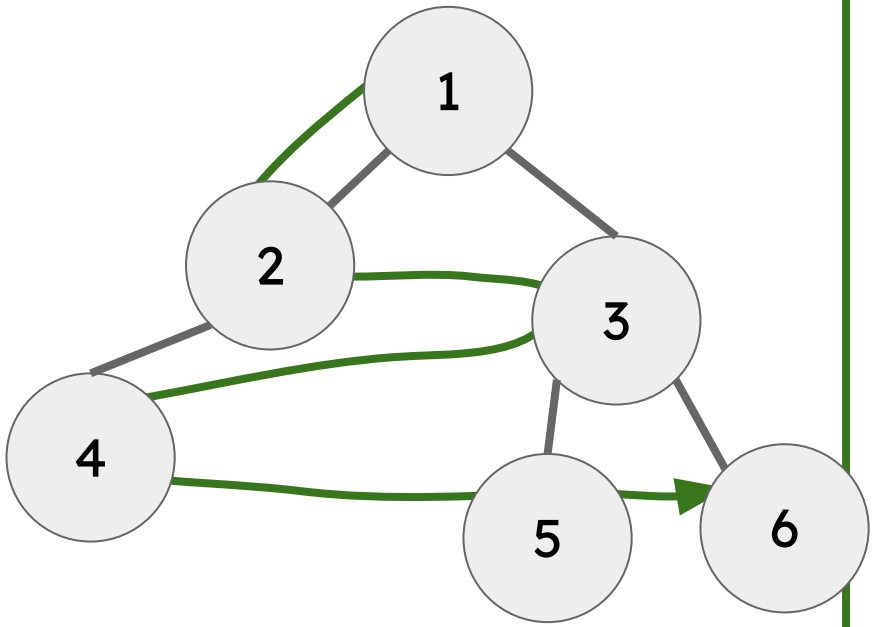


BFS:

What about space usage? What type of graphs use less space on DFS than BFS? What type of graphs use less space on BFS than DFS?



DFS:



BFS:

Something to Try

Adapting Algorithms:

Showing you a fixed version of BFS/DFS isn't very helpful, I **don't encourage you to hard memorise** any version of BFS or DFS.

Depending on the problem you're facing, you might have to either modify internals, API, termination conditions, what we return, and so on.

The rough idea of a traversal remains, but there's so much that can be changed about it.

Adapting Algorithms:

You might also just make use of existing solutions, but change their outputs.

E.g. run the flood-fill algorithm, it basically also solves component counting!

Next Lecture:

We will see how to adapt the DFS algorithm to solve even more problems!